



Arora V Design Physical Constraints User Guide

SUG1018-1.7E, 08/29/2025

Copyright © 2025 Guangdong Gowin Semiconductor Corporation. All Rights Reserved.

GOWIN and GOWIN are trademarks of Guangdong Gowin Semiconductor

Corporation and are registered in China, the U.S. Patent and Trademark Office, and other countries. All other words and logos identified as trademarks or service marks are the property of their respective holders. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior written consent of GOWINSEMI.

Disclaimer

GOWINSEMI assumes no liability and provides no warranty (either expressed or implied) and is not responsible for any damage incurred to your hardware, software, data, or property resulting from usage of the materials or intellectual property except as outlined in the GOWINSEMI Terms and Conditions of Sale. GOWINSEMI may make changes to this document at any time without prior notice. Anyone relying on this documentation should contact GOWINSEMI for the current documentation and errata.

Revision History

Date	Version	Description
04/20/2023	1.0E	Initial version published.
06/30/2023	1.1E	JTAGSEL_N Net, JTAG Net, SSPI Net, MODE Net, and I2C Net constraints added.
08/18/2023	1.2E	● The descriptions of JTAG Net, SSPI Net, MODE Net, and I2C Net constraints updated. ● The MIPI_DPHY_RX primitive constraint example added. ● Timing Paths function removed.
11/30/2023	1.2.1E	● Some screenshots in Chapter 3 FloorPlanner and Chapter 4 Using FloorPlanner updated. ● The title description for the dialog box that pops up when "Instance" selected in "HCLK/GCLK Primitive Constraints" updated.
02/02/2024	1.2.2E	● Descriptions of global clock constraints and high-speed clock constraints related to GW5A(T)-60 added. ● Figure 3-22 Back-annotate Port updated.
05/09/2024	1.3E	● Descriptions of IO attribute Ctle added. ● RECONFIG_N Net constraints added. ● Descriptions of Primitive Group constraint supporting wildcards added.
06/28/2024	1.4E	● Figure 3-21 Back-annotate Physical Constraints updated. ● Descriptions of global clock and high-speed clock constraints for GW5A(N)(R)T-15 added. ● Descriptions of IO attribute Slew Rate added.
08/09/2024	1.5E	● Descriptions of DSPLite and SCRAMBLE primitives for GW5A(N)(R)T-15 added. ● Descriptions of IO attribute Pull Strength added. ● Descriptions of IO attribute Vref updated. ● Reset Layout option added. ● Figure 3-23 View Menu and Figure 3-24 Windows Menu updated. ● Descriptions of the interface layout memory feature in 3.2 Start FloorPlanner added. ● Descriptions of 2.10.1 ADC Input Voltage Source Constraint updated.
06/27/2025	1.6E	Descriptions of Chapter 2 Physical Constraints Syntax Definition updated.
08/29/2025	1.7E	2.10.8 NET_NO_REP Constraints added.

Contents

Contents	i
List of Figures	iii
List of Tables	vi
1 About This Guide	1
1.1 Purpose	1
1.2 Related Documents	1
1.3 Terminology and Abbreviations	2
1.4 Support and Feedback	3
2 Physical Constraints Syntax Definition	4
2.1 I/O Location Constraints	4
2.2 I/O Attribute Constraints	5
2.3 Primitive Location Constraints	6
2.4 Group Constraints	10
2.4.1 Primitive Group Constraints	10
2.4.2 Relative Group Constraints	12
2.5 Resource Reservation Constraints	13
2.6 Vref Constraints	13
2.7 Clock Net Constraints	14
2.8 GCLK Primitive Constraints	16
2.9 HCLK Primitive Constraints	17
2.10 Other Constraints	18
2.10.1 ADC Input Voltage Source Constraint	18
2.10.2 JTAGSEL_N Net Constraints	19
2.10.3 RECONFIG_N Net Constraints	20
2.10.4 JTAG Net Constraints	20
2.10.5 SSPI Net Constraints	21
2.10.6 MODE Net Constraints	22
2.10.7 I2C Net Constraints	23
2.10.8 NET_NO_REP Constraints	24

3 FloorPlanner	26
3.1 Introduction	26
3.2 Start FloorPlanner	27
3.3 FloorPlanner Interface	28
3.3.1 Menu Bar	29
3.3.2 Summary and Netlist Windows	39
3.3.3 Package View	43
3.3.4 Chip Array Window	47
3.3.5 Constraints Editing Window	53
3.3.6 Message Window	58
4 Using FloorPlanner	60
4.1 Create Constraint File	60
4.2 Edit Constraints File	62
4.2.1 Constraints Examples	62
4.2.2 Edit I/O Constraints	64
4.2.3 Edit Primitive Constraints	65
4.2.4 Edit Group Constraints	66
4.2.5 Edit Resource Reservation Constraints	70
4.2.6 Edit Clock Net Constraints	71
4.2.7 Edit GCLK Primitive Constraints	72
4.2.8 Edit HCLK Primitive Constraints	73
4.2.9 Edit Vref Constraints	74

List of Figures

Figure 3-1 Start FloorPlanner via Menu Bar	27
Figure 3-2 Start FloorPlanner in Process View.....	27
Figure 3-3 Start FloorPlanner via Start Page.....	28
Figure 3-4 FloorPlanner Interface	29
Figure 3-5 File	29
Figure 3-6 Open Physical Constraints	30
Figure 3-7 Constraints	30
Figure 3-8 Select Primitive	31
Figure 3-9 New Primitive Group.....	32
Figure 3-10 Right Primitive Group	32
Figure 3-11 Invalid Locations	33
Figure 3-12 Invalid Locations	33
Figure 3-13 New Relative Group	34
Figure 3-14 Right Relative Group	34
Figure 3-15 Resource Reservation	34
Figure 3-16 Clock Assignment	35
Figure 3-17 GCLK Primitive Constraints.....	36
Figure 3-18 HCLK Primitive Constraints.....	37
Figure 3-19 Vref Constraints	37
Figure 3-20 Tools	37
Figure 3-21 Back-annotate Physical Constraints.....	38
Figure 3-22 Back-annotate Port.....	38
Figure 3-23 View Menu	39
Figure 3-24 Windows Menu	39
Figure 3-25 Summary Window	40
Figure 3-26 Netlist View	41
Figure 3-27 BUS and Non-Bus Display	42
Figure 3-28 Hierarchy Display	42
Figure 3-29 Netlist Right-clicking	43
Figure 3-30 Package View (GW5AT-138-FPBGA676A).....	44

Figure 3-31 Package View Right-clicking	45
Figure 3-32 Differential Pair Display	45
Figure 3-33 Top View	46
Figure 3-34 Bottom View	46
Figure 3-35 Chip Array Window	47
Figure 3-36 Constraints in Grid	48
Figure 3-37 Constraints in Macrocell	48
Figure 3-38 Constraints in Primitive	49
Figure 3-39 Chip Array Right-clicking	51
Figure 3-40 Show Place View	51
Figure 3-41 Mouse Hovering Display	52
Figure 3-42 Right-click to Select Highlight	53
Figure 3-43 I/O Constraints View	55
Figure 3-44 Primitive Constraints View	55
Figure 3-45 Group Constraints View	56
Figure 3-46 Resource Reservation View	56
Figure 3-47 Clock Assignment View	57
Figure 3-48 GCLK Primitive Constraints	57
Figure 3-49 HCLK Primitive Constraints	58
Figure 3-50 Vref Constraints View	58
Figure 3-51 Message Window	59
Figure 4-1 New Physical Constraints	60
Figure 4-2 Select Device	61
Figure 4-3 Save Output File	62
Figure 4-4 Drag to Chip Array to Create I/O Constraints	64
Figure 4-5 Drag to Package View to Create I/O Constraints	65
Figure 4-6 Drag to Chip Array to Create Primitive Constraints	66
Figure 4-7 Group Constraints Right-clicking	66
Figure 4-8 Create Primitive Group Constraints	67
Figure 4-9 Primitive Group Constraints	68
Figure 4-10 Create Relative Group Constraints	69
Figure 4-11 Relative Group Constraints	70
Figure 4-12 Create Resource Reservation	70
Figure 4-13 Resource Reservation	71
Figure 4-14 Create Clock Assignment Constraints	71
Figure 4-15 Clock Assignment Constraints	72
Figure 4-16 Create GCLK Primitive Constraints	72
Figure 4-17 GCLK Primitive Constraints	73

Figure 4-18 Create HCLK Primitive Constraints	73
Figure 4-19 HCLK Primitive Constraints	74
Figure 4-20 Create Vref Constraints	74
Figure 4-21 Drag to Chip Array to Generate Vref Constraints Location	75
Figure 4-22 Drag to Package View to Generate Vref Constraints Location	76
Figure 4-23 Duplicate Names Prompt	77

List of Tables

Table 1-1 Terminology and Abbreviations	2
Table 2-1 Constraint Positions for DCS /DCE.....	16
Table 2-2 Constraint Positions for CLKDIV/DLLDLY	18

1 About This Guide

1.1 Purpose

This manual describes the interface and use of FloorPlanner and the syntax definition of physical constraint for Arora V FPGA products in order to help you add physical constraints to your design. As the software is subject to change without notice, some information may not remain relevant and may need to be adjusted according to the software that is in use.

All physical constraints have higher priority than the global Place & Route settings in Gowin Software. The global Place & Route settings can be configured via clicking "Project > Configuration > Place & Route" from the menu bar in Gowin Software.

1.2 Related Documents

The latest user guides are available on GOWINSEMI Website: www.gowinsemi.com. You can find the related documents:

- [SUG100, Gowin Software User Guide](#)
- [UG306, Arora V Clock User Guide](#)
- [UG704, Arora V 138K FPGA Product Programming and Configuration Guide](#)
- [UG714, Arora V 25K FPGA Products Programming and Configuration Guide](#)
- [UG718, Arora V 60K FPGA Products Programming and Configuration Guide](#)
- [UG720, Arora V 15K FPGA Products Programming and Configuration Guide](#)

- [DS981, GW5AT series of FPGA Products Data Sheet](#)
- [DS1103, GW5A series of FPGA Products Data Sheet](#)
- [DS1239, GW5AST series of FPGA Products Data Sheet](#)
- [DS1105, GW5AS series of FPGA Products Data Sheet](#)
- [DS1108, GW5AR series of FPGA Products Data Sheet](#)
- [DS1118, GW5ART series of FPGA Products Data Sheet](#)

1.3 Terminology and Abbreviations

Table 1-1 shows the abbreviations and terminology that are used in this manual.

Table 1-1 Terminology and Abbreviations

Terminology and Abbreviations	Meaning
BSRAM	Block SRAM
CFU	Configurable Function Unit
CLKDIV	Clock Divider
CLS	Configurable Logic Section
DCS	Dynamic Clock Selector
DDRDLL	Double Data Rate Delay-locked Loop
DLLDLY	DLL Delay
DQS	Bidirectional Data Strobe Circuit for DDR Memory
FloorPlanner	Physical Constraint Editor
FPGA	Field Programmable Gate Array
GCLK	Global Clock
I/O	Input/Output
IDE	Integrated Development Environment
LUT	Look-up Table
LW	Long Wire
PLL	Phase-locked Loop
SSRAM	Shadow SRAM
VREF	Voltage Reference

1.4 Support and Feedback

Gowin Semiconductor provides customers with comprehensive technical support. If you have any questions, comments, or suggestions, please feel free to contact us directly by the following ways.

Website: www.gowinsemi.com

E-mail: support@gowinsemi.com

2 Physical Constraints Syntax Definition

2.1 I/O Location Constraints

The IO constraints can be used to constrain ports to the specified IO locations. The specific IO locations can be found in the device's [Pinout manuals](#).

Syntax

```
IO_LOC "obj_name" obj_location [exclusive];
```

Syntax Rules

- Each constraint statement must end with an English semicolon ";".
- The constraint object `obj_name` must be enclosed in English double quotation marks.
- In a constraint statement, multiple constraint locations (`obj_location`) can be specified, separated by English commas. For example, multiple IO locations can be written as "A11,B2".
- Each part of the constraint statement must be separated by a space.

Constraint Elements

- `obj_name`

`obj_name` is the name of the port in the example.

- `obj_location`

`obj_location` is the IO location, such as "A11", "B12", etc. If multiple locations are specified, they need to be separated by commas, such as "A11, B2".

- **exclusive**

exclusive is optional and follows the constraint location, indicating that only the object specified by `obj_name` can be placed in the `obj_location` of the constraint statement.

Examples

Example 1

```
IO_LOC "port0" G2;
```

// The object `port0` is constrained to the location `G2` in the `GW5AT-15B-MBGA132` package.

Example 2

```
IO_LOC "port1" P10,N10,G2;
```

// The object `port1` is constrained to locations `P10`, `N10`, and `G2` in the `GW5AT-15B-MBGA132` package. During placement, the first valid location among the three will be used.

Example 3

```
IO_LOC "port2" P10,N10,G2 exclusive;
```

// The object `port2` is constrained to `P10`, `N10`, and `G2` in the `GW5AT-15B-MBGA132` package, and these three locations can only be used by `port2`.

2.2 I/O Attribute Constraints

You can set attribute values for I/O, such as `IO_TYPE`, `PULL_MODE` and `DRIVE`, etc. For detailed attribute settings, please refer to the product [datasheets](#).

Syntax

```
IO_PORT " obj_name " attribute = attribute_value;
```

Note !

- Multiple attributes can be specified in a single constraint statement, with each attribute separated by a space.
- For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Elements

- **obj_name**

`obj_name` is the name of the port in the example.

- attribute
Name of the IO attribute
- attribute value
The value of the IO attribute

For the details of IO attribute and attribute value, you can see [UG304, Arora V Programmable IO \(GPIO\) User Guide](#).

Examples

Example 1

```
IO_PORT "port_1" IO_TYPE = LVTTL33;
// Sets port_1 IO_TYPE to "LVTTL33".
```

Example 2

```
IO_PORT "port_2" IO_TYPE = LVTTL33 PULL_MODE = KEEPER;
// Sets port_2 IO_TYPE to the LVTTL33; PULL_MODE value is set to
"KEEPER".
```

2.3 Primitive Location Constraints

Primitive Constraints are used to constrain the primitives to the specified locations.

LUT/DFF/DL/MUX/IOLOGIC/BSRAM/SSRAM/DSP/DDRDLL/PLL/DQS/MI
PI_DPHY_RX/
DSPLite/SCRAMBLE instances can be constrained using Primitive Constraints.

Syntax

```
INS_LOC "obj_name" obj_location [exclusive];
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Elements

- obj_name
The constrained object, which refers to the instance name of the primitive.
- exclusive
The keyword "exclusive" is optional. It indicates that the obj_location

specified in this constraint statement can only be used for placing the object defined by `obj_name`.

- `obj_location`

The valid location(s) for the primitive specified by `obj_name`. `obj_location` includes the following categories:

1. LUT Constraint Location

- A signal location specifying to LUT, such as, `RxCy[0-3][A-B]`.
- A range of the locations specifying multiple rows or columns.
 - Includes multiple CLSs or LUTs: `"RxCm"`, `"RxCm[0-3]"`
 - Specifies multiple rows: `"R[x:y]Cm"`, `"R[x:y]Cm[0-3]"`, `"R[x:y]Cm[0-3][A-B]"`
 - Specifies multiple columns: `"RxC[m:n]"`, `"RxC[m:n][0-3]"`, `"RxC[m:n][0-3][A-B]"`
 - Specifies multiple rows and columns: `"R[x:y]C[m:n]"`, `"R[x:y]C[m:n][0-3]"`, `"R[x:y]C[m:n][0-3][A-B]"`

Note!

- `x`, `y` in the LUT location refers to the row information of the GRID.
- `m`, `n` in the LUT location refers to the column information of the GRID.
- `R` in the LUT location represents the row, and `C` represents the column of the GRID.
- `0-3` in the LUT represents the index of the CLS within a GRID.
- `A-B` in the LUT location represents the index of LUT location within a CLS.
- For MUX, DFF, and DL primitives, the constraint locations are the same as those for LUTs.

2. PLL Constraint Location

The format for PLL constraint locations is either `PLL_L` or `PLL_R`, representing the left or right side of the device. If multiple PLLs are available on one side, they can be indexed as:

`PLL_L[0]`, `PLL_L[1]`, ...

`PLL_R[0]`, `PLL_R[1]`, ...

3. BSRAM Constraint Location

The writing format for BSRAM constraint location is `"BSRAM_R10[0]"` (the first BSRAM at row 10), `"BSRAM_R10[1]"`....

4. DSP Constraints Location

The format for DSP constraint locations is DSP_R37[0] (the first DSP block in row 37), DSP_R37[1], ...

If it is constrained to a macro, the format can be written as DSP_R37[0][A] or DSP_R37[0][B].

Note!

Each DSP block contains two macros. A macro refers to the location for one MULT18X18 primitive.

5. DDRDLL Constraint Location

The writing format for DDRDLL constraint location is "DDRDLLM_TL", "DDRDLLM_TR".

6. MIPI_DPHY_RX Constraint Location

The writing format for MIPI_DPHY_RX constraint location is "QUAD[0]", "QUAD [1]".

Note!

MIPI_DPHY_RX constraints support the following devices: GW5A(S)(T)-138, but do not support exclusive.

7. DSPLite Constraint Location

The writing format for DSPLite constraint location is "R19C26", "R19C37"...,"R19C47".

Note!

DSPLite constraints support the following devices: GW5A(N)(R)T-15.

8. SCRAMBLE Constraint Location

The writing format for SCRAMBLE constraint location is "SCRM[0]", "SCRM[1]"...

Note!

SCRAMBLE constraints support the following devices: GW5A(N)(R)T-15, but the exclusive option is not supported.

9. IOLOGIC Constraint Location

IOLOGIC constraint locations refer to specific IO positions. Please refer to the [Pinout manuals](#) of the target device for detailed IO location information.

Examples

Example 1

```
INS_LOC "lut_1" R5C10[0][A];
```

// lut_1 is constrained at the 1st LUT of the 1st CLS of R5C10.

Example 2

```
INS_LOC "ins_2 " R5C6[2] exclusive;
```

// ins_2 is constrained at the 3rd CLS of R5C6, and only this constrained object can be placed in this location.

Example 3

```
INS_LOC "ins_3" R[2:6]C2;
```

// ins_3 is constrained at the range between second to sixth rows and the second column.

Example 4

```
INS_LOC "ins_4" R[2:4]C[2:6] exclusive;
```

// ins_4 is constrained at the range between the second to fourth rows and the second to sixth columns, and this range can only be used by this constrained object.

Example 5

```
INS_LOC "ins_5" R[2:4]C[2:6][1];
```

// ins_5 is constrained at the 2nd CLS of any GRID in the range between the second to fourth rows and the second to sixth columns.

Example 6

```
INS_LOC "mipi_dphy_rx_name" QUAD[0];
```

// In the GW5AT-138B-PBGA484 package, constrain mipi_dphy_rx_name to the first MIPI_DPHY_RX location.

Example 7

```
INS_LOC "scramble_name" SCRM[0];
```

// In the GW5AT-15B-MBGA132 package, constrain scramble_name to the first SCRAMBLE location.

Example 8

```
INS_LOC "ides4_name" P10;
```

// In the GW5AT-15B-MBGA132 package, constrain ides_name to IO package location P10.

Example 9

```
INS_LOC "pll_name" PLL_L[0];
```

// In the GW5AST-138B-PBGA484A package, constrain pll_name to the first PLL location on the left side.

Example 10

```
INS_LOC "bsram_name" BSRAM_R10[2];
```

// In the GW5AST-138B-PBGA484A package, constrain bsram_name to the third BSRAM location in row 10.

Example 11

```
INS_LOC "dsp_name" DSP_R37[2];
```

// In the GW5AST-138B-PBGA484A package, constrain dsp_name to the third DSP block in row 37.

Example 12

```
INS_LOC "ddrdll_name" DDRDLLM_TL;
```

// In the GW5AST-138B-PBGA484A package, constrain ddrdll_name to the DDRDLL location at the top-left.

2.4 Group Constraints

The group constraints include Primitive Group Constraints and Relative Group Constraints.

2.4.1 Primitive Group Constraints

Primitive Group Constraint is used to define a group constraint. A group is a collection of various instance objects. The instances such as LUT, DFF, DL, IOLOGIC, Buffer, BSRAM, SSRAM, DSP, PLL, DDRDLL, DQS, DSPLite, or Buffer, IOLOGIC can be added to a group using the Primitive Group constraints. And the location constraints of all objects in the group can be achieved by constraining the location of the group. At the same time, Primitive Group constraints support the use of wildcard characters "?" and "*", where "?" matches a single character and "*" matches zero or more characters.

Syntax

Primitive Group Constraint Definition:

```
GROUP group_name = { "obj_names" } [exclusive];
```

GRP_LOC group_name group_location[exclusive];

Note!

- For syntax rules, you can see [Syntax Rules](#) in Section 2.1.
- The primitive group constraint syntax supports the use of wildcards: "?" matches a single character, and "*" matches zero or more characters.
- The primitive group constraint syntax consists of two statements: group definition and group location. If either is missing, the constraint is considered invalid.

Constraint Elements

- group_name

Defines the name of a Primitive Group.

- obj_name

obj_name refers to the instance object added to the Primitive Group.

- group_location

Specifies the constraint locations for the group, such as group_location can be an IO, CFU, DQS, BSRAM, DSP, PLL, DDRDLL, or DSPLite location.

- exclusive

The keyword "exclusive" is optional, which is placed at the end of the group definition or the group location constraints.

An object can be included in multiple groups, but the object can only be included in the group that the "exclusive" is added.

The "exclusive" indicates that the constraints location can only be used by the objects in the group.

Examples

Example 1

```
GROUP group_1 = { "lut1" "lut2" "lut3" "ins_4" };
```

```
GRP_LOC group_1 R[3:4]C4 exclusive;
```

// Creates a group named group_1, adding the objects lut1, lut2, lut3, and ins_4 to this group. All instances in the group are allowed to be placed within the region R[3:4]C4, and this region is exclusively reserved for instances in group_1.

Example 2

```
GROUP group_2 = { "lut_*" } exclusive;
```

```
GRP_LOC group_2 R[2:3]C[2:4];
```

// This example demonstrates the use of the wildcard *. A group named group_2 is created, which includes all instances whose names start with lut_, and all instances are exclusively assigned to this group. The instances can be placed within the region R[2:3]C[2:4].

2.4.2 Relative Group Constraints

The instances such as LUT1~LUT4, DFF, DL, MUX2 relative location constraints can be realized using the Relative Group Constraints.

Syntax

Relative Group Constraint Definition:

```
REL_GROUP group_name = { "obj_names" };
```

```
INS_RLOC "obj_name" relative_location;
```

Note!

- For syntax rules, you can see [Syntax](#) in Section 2.1.
- The relative group constraint syntax consists of two statements: group definition and group location. If either is missing, the constraint is considered invalid.

Constraint Elements

- obj_name

Defines the name of a Relative Group.

- relative_location

Describes the relative location information of each constrained object within the group.

Example

```
REL_GROUP grp_1 = { "ins_1" "ins_2" "ins_3" };
```

```
INS_RLOC "ins_1" R1C1;
```

```
INS_RLOC "ins_2" R2C3;
```

```
INS_RLOC "ins_3" R0C5;
```

// Defines a relative group constraint named grp_1 and adds the ins_1, ins_2, ins_3 to grp_1. Use the relative location R1C1 of ins_1 as the origin. Constrain ins_2 to the relative location R2C3 from R1C1, and ins_3 to the relative location R0C5 from R1C1.

2.5 Resource Reservation Constraints

The specified location or range can be reserved using the Resource Reservation constraints.

Syntax

```
LOC_RESERVE location [ -res_obj ];
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Elements

- location

Reserved location information. Only locations of the following types can be reserved: CFU, IO, BSRAM, DSP, DDRDLL, PLL, DQS, MIPI_DPHY_RX, DSPLite, and SCRAMBLE.

- res_obj

It is optional, and it is only valid for CFU type in location. It supports two values: LUT and REG. Setting it to LUT means reserving only the LUT resources in the specified location, while setting it to REG meaning reserving only the REG resources.

Examples

Example 1

```
LOC_RESERVE R2C3[0]-LUT;
```

```
// The LUT resources in the first CLS of R2C3 are reserved.
```

Example 2

```
LOC_RESERVE R2C3[0]-REG;
```

```
// The REG resources in the first CLS of R2C3 are reserved.
```

Example 3

```
LOC_RESERVE IOR3, IOR6, R2C3, R3C4;
```

```
// The resources at locations IOR3, IOR6, R2C3, and R3C4 are reserved and cannot be used during the placement.
```

2.6 Vref Constraints

The chip supports the external reference voltage, which is valid for the

BANK. The Vref constraints can be used to constrain the name and location of the input pin of the external reference voltage.

Note!

- The input pin location where the external reference voltage can be set must have IOLOGIC resource.
- Vref Constraints and PORT constraints are valid when used together. When a single-ended input or inout port with IO Type SSTL/HSTL, the Vref attribute can be set to the created Vref Constraints, indicating that the reference voltage of this port uses the external reference voltage input from the Vref Constraints location.

Syntax

```
USE_VREF_DRIVER vref_name [location];
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Elements

- vref_name
User-defined VREF pin name.
- location

Any I/O location containing IOLOGIC resources can be used as the location for the VREF pin constraint.

Example

```
USE_VREF_DRIVER vref_pin L14;
```

```
IO_LOC "port_1" L13;
```

```
IO_PORT "port_1" IO_TYPE = SSTL18_I VREF = vref_pin;
```

// In GW5AT-15B-MBGA132, define a VREF pin named vref_pin, and set the VREF attribute of port_1 to vref_pin. The constrained locations L13 and L14 are in the same bank.

2.7 Clock Net Constraints

The Clock Net Constraints is used to constrain a specific net to the global clock wire or the non-clock wire in the design.

Syntax

```
CLOCK_LOC "net_name" global_clocks = signal_type;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Elements

● net_name

The name of the net used in the example.

● global_clocks

Specifies the clock type that the net routes.

BUFG[0-15] means this net is routed to a specific global clock wire.

BUFG means this net is routed to the global clock wire.

LOCAL_CLOCK means this net is not routed to the global clock wire.

● signal_type

Specifies the signal type of the net.

CLK: The net functions as a clock pin.

CE: The net functions as a clock enable pin.

SR: The net functions as SET, RESET, CLEAR, or PRESET signals.

LOGIC: The net does not fall into any of the above categories.

The sign "|" can be used to separate multiple specified signal_type.

Note!

If LOCAL_CLOCK is selected for global_clocks, signal_type is not available.

Examples

Example 1

```
CLOCK_LOC "net" BUFG = CLK|CE;
```

```
NET_LOC "net" BUFG = CLK|CE;
```

// Constrains the clock net with signal_type set to clock pin or clock enable pin to be routed through the global clock wire.

Example 2

```
CLOCK_LOC "net" LOCAL_CLOCK;
```

// Constrains the net to be routed through local (non-clock) wiring.

Example 3


```
CLOCK_LOC "net" BUFG[0] = CLK;
```

// Constrains the clock net with signal_type set to clock pin to be routed through the first global clock wire.

2.8 GCLK Primitive Constraints

GCLK Primitive Constraints are used to constrain the DCS, DCE to the specified locations.

Syntax

```
INS_LOC "obj_name" position;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Element

- **obj_name**

Uses the instance name of DCS or DCE as obj_name.

- **Position**

The position of the constrained object.

Table 2-1 Constraint Positions for DCS /DCE

Device	Position	
	DCE	DCS
GW5A(S)(T)-138/ GW5AT-75	PTR0, PTR1, PTR2, PTR3, PTR0[0~5], PTR1[0~5], PTR2[0~5], PTR3[0~5], PBR0, PBR1, PBR2, PBR3, PBR0[0~5], PBR1[0~5], PBR2[0~5], PBR3[0~5], PG, SG, PG[0~11], SG[0~15]	PTR0, PTR1, PTR2, PTR3, PTR0[0~1], PTR1[0~1], PTR2[0~1], PTR3[0~1], PBR0, PBR1, PBR2, PBR3, PBR0[0~1], PBR1[0~1], PBR2[0~1], PBR3[0~1], PG, PG[0~3]
GW5A(R)(S)-25	TOPLEFT, TOPRIGHT, BOTTOMLEFT, BOTTOMRIGHT, STOP, SBOTTOM	TOPLEFT, TOPRIGHT, BOTTOMLEFT, BOTTOMRIGHT
GW5A(T)- 60	PG, SG, PG[0~5], SG[0~7]	PG, PG[0~1]
GW5A(N)(R)T-15	PG, SG, PG[0~5], SG[0~7]	PG, PG[0~1]

Note!

The specific location of the constraint position for DCS/DCE, see [UG306, Arora V Clock User Guide](#).

Example

```
INS_LOC "dcs_name" PTR0[1].
```

// Constrains the DCS object dcs_name to the PTR0[1] position in the GW5AT-138B-PBGA484A package.

2.9 HCLK Primitive Constraints

The CLKDIV/DLLDLY can be constrained to the Hclk locations using the HCLK Primitive Constraints. The constraints locations of CLKDIV/DLLDLY are different from those of other general instances. The "BOTTOMSIDE", "LEFTSIDE", and "RIGHTSIDE" indicate the sides of the constraints location.

Syntax

```
INS_LOC "obj_name" position;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraints Elements

- **obj_name**
Uses the instance name of CLKDIV or DLLDLY as obj_name.
- **Position**
The position of the constrained object.

Table 2-2 Constraint Positions for CLKDIV/DLLDLY

Device	Position	
	CLKDIV	DLLDLY
GW5A(S)(T)-138/ GW5AT-75	BOTTOMSIDE[0~7], LEFTSIDE[0~7], RIGHTSIDE[0~7]	BOTTOMSIDE[0~3], LEFTSIDE[0~3], RIGHTSIDE[0~3]
GW5A(R)(S)-25	LEFTSIDE[0~3], RIGHTSIDE[0~3], BOTTOMSIDE[0~3], TOPSIDE[0~3]	BOTTOMSIDE[0~1], LEFTSIDE[0~1], RIGHTSIDE[0~1], TOPSIDE[0~1]
GW5A(T)-60	LEFTSIDE[0~3], RIGHTSIDE[0~3], BOTTOMSIDE[0~7], TOPSIDE[0~3]	BOTTOMSIDE[0~3], LEFTSIDE[0~1], RIGHTSIDE[0~1], TOPSIDE[0~1]
GW5A(N)(R)T-15	LEFTSIDE[0~3], BOTTOMSIDE[0~3]	BOTTOMSIDE[0~1], LEFTSIDE[0~1]

Example

```
INS_LOC "clkdiv_name" LEFTSIDE[0];
```

// Constrains the clkdiv_name to LEFTSIDE[0] in the GW5AT-138B-PBGA484A package.

2.10 Other Constraints

2.10.1 ADC Input Voltage Source Constraint

The ADC input voltage source can come from external I/Os. You can specify the points at which the ADC receives the input voltage source by constraining the IO locations.

Note!

- Supported devices for ADC input voltage constraints: All devices support this function except for GW5A(S)(T)-138 and GW5AT-75.
- For the IO locations for each bus on different devices, you can see the pinout manual.

Syntax

```
USE_ADC_SRC bus0~n location;
```

Note!

For syntax rules, you can see [Syntax](#) in Section 2.1.

Constraint Element

- bus0~n

The configurable values of 0~n can be found in the "ADC_INPUT" column of the Pinout manual for each device.

- location

The location refers to an I/O location. Only the location format listed under the "Pin Name" column in the Pinout Manual is supported. Each constraint must apply to an entire differential pair location, such as IOT48.

Example

```
USE_ADC_SRC bus2 IOT97
```

```
// Sets IOT97 as the external input location for the ADC input voltage  
source in the GW5AT-60B-UBGA324A package.
```

2.10.2 JTAGSEL_N Net Constraints

When the internal logic of the FPGA is used to control JTAGSEL_N, i.e., during the second downloading without power off, pull down JTAGSEL_N so that JTAG can be switched to the configuration function, and it is necessary to add net physical constraints for JTAGSEL_N.

Syntax

```
NET_LOC "obj_name" V_JTAGSELN;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Element

- obj_name

The net that can be wired in the internal logic is used as obj_name.

Example

```
NET_LOC "netname" V_JTAGSELN;
```

```
//The netname net is used to control the functions of JTAGSEL_N.
```

2.10.3 RECONFIG_N Net Constraints

When the internal logic of the FPGA is used to control the RECONFIG_N , i.e., during the second downloading without power off, pull down RECONFIG_N to switch FPGA to configuration reset mode, and it is necessary to add net physical constraints for RECONFIG_N.

Syntax

```
NET_LOC "obj_name" V_RECONFIGN;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Element

- obj_name

The net that can be wired in the internal logic is used as obj_name.

Example

```
NET_LOC "netname" V_RECONFIGN;
```

```
//The netname net is used to control the functions of RECONFIG_N.
```

2.10.4 JTAG Net Constraints

The JTAG function pins include TCK, TMS, TDI, TDO, which can be realized through IO configuration or FPGA internal logic control, and these two methods are mutually exclusive; these function pins can only be realized in the same method, i.e., selecting the IO configuration to realize these pin functions or selecting the FPGA internal logic control to realize them; in addition, TDO can only be realized through IO configuration and is not affected by the implementation mode of TCK, TMS and TDI.

When the internal logic of the FPGA is used to control TCK, TMS, and TDI of JTAG, it is necessary to add the net physical constraints for JTAG; for the details, you can see Arora V FPGA product programming and configuration guide.

- [UG704, Arora V 138K FPGA Product Programming and Configuration Guide](#)
- [UG714, Arora V 25K FPGA Products Programming and Configuration Guide](#)
- [UG718, Arora V 60K FPGA Products Programming and Configuration](#)

[Guide](#)

- [UG720, Arora V 15K FPGA Products Programming and Configuration Guide](#)

Syntax

```
NET_LOC " obj_Name" V_TCK;  
NET_LOC " obj_Name" V_TMS;  
NET_LOC " obj_Name" V_TDI;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Element

- obj_name

The net that can be wired in the internal logic is used as obj_name.

Example

```
NET_LOC "netname0" V_TCK;  
NET_LOC "netname1" V_TMS;  
NET_LOC "netname2" V_TDI;
```

// The netname0 net is used to control the functions of TCK; the netname1 is used to control the functions of TMS, and netname2 is used to control the functions of TDI.

2.10.5 SSPI Net Constraints

SSPI pins include SI, SO, SSPI_WPN, CLKHOLD_N, SSPI_CLK, and SSPI_CS_N, which can be realized through IO configuration or FPGA internal logic control, and these two methods are mutually exclusive; these function pins can only be realized in the same method, i.e., selecting the IO configuration to realize these pin functions or selecting the FPGA internal logic control to realize them.

When the internal logic of the FPGA is used to control SSPI functions, it is necessary to add the net physical constraints for SSPI; for the details, you can see Arora V FPGA product programming and configuration guide.

Syntax

```
NET_LOC " obj_Name" V_SSPI SI;
```

```
NET_LOC " obj_Name" V_SSPIISO;  
NET_LOC " obj_Name" V_SSPIWPN;  
NET_LOC " obj_Name" V_SSPICLKHOLDN;  
NET_LOC " obj_Name" V_SSPICLK;  
NET_LOC " obj_Name" V_SSPICSN;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Element

- obj_name

The net that can be wired in internal logic is used as obj_name.

Examples

```
NET_LOC "netname0" V_SSPISI;  
NET_LOC "netname1" V_SSPIISO;  
NET_LOC "netname2" V_SSPIWPN;  
NET_LOC "netname3" V_SSPICLKHOLDN;  
NET_LOC "netname4" V_SSPICLK;  
NET_LOC "netname5" V_SSPICSN;
```

//The netname0 is used to control the SI function; the netname1 is used to control the SO function; the netname2 is used to control the SSPI_WPN function; the netname3 is used to control the CLKHOLD_N function; the netname4 is used to control the SSPI_CLK function, and the netname5 is used to control the SSPI_CS_N function.

2.10.6 MODE Net Constraints

MODE pins include MODE0, MODE1, MODE2, which can be realized through IO configuration or FPGA internal logic control, and these two methods are mutually exclusive; these function pins can only be realized in the same method, i.e., selecting the IO configuration to realize these pin functions or selecting the FPGA internal logic control to realize them.

When the FPGA internal logic is used to control the function of MODE, in addition to MODE0, MODE1 and MODE2, the MODE_LD signal is added, which is a loading signal that loads the value of MODE0~MODE2

controlled by the internal logic on the rising edge, i.e., switching the value of MODE0~MODE2 when MODE_LD is at a rising edge; prior to the first rising edge of MODE_LD, the values set for MODE0 to MODE2 during bitstream download are used. When the internal logic of the FPGA is used to control MODE functions, it is necessary to add the net physical constraints for MODE; for the details, you can see Arora V FPGA product programming and configuration guide.

Syntax

```
NET_LOC " obj_Name" V_MODE0;  
NET_LOC " obj_Name" V_MODE1;  
NET_LOC " obj_Name" V_MODE2;  
NET_LOC "obj_Name" V_MODE_LD;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Element

- obj_name

The net that can be wired in the internal logic is used as obj_name.

Examples

```
NET_LOC "netname0" V_MODE0;  
NET_LOC "netname1" V_MODE1;  
NET_LOC "netname2" V_MODE2;  
NET_LOC "netname3" V_MODE_LD;
```

// The netname0 is used to control the MODE0 function; the netname1 is used to control the MODE1 function; the netname2 is used to control the MODE2 function, and the netname3 is used to control the MODE_LD function.

2.10.7 I2C Net Constraints

The I2C pins include SCL and SDA, which can be realized through IO configuration or FPGA internal logic control, and these two methods are mutually exclusive; these function pins can only be realized in the same method, i.e., selecting the IO configuration to realize these pin functions or selecting the FPGA internal logic control to realize them.

When the internal logic of the FPGA is used to control I2C functions, it is necessary to add the net physical constraints for I2C.

Syntax

```
NET_LOC " obj_Name" V_SCL;
```

```
NET_LOC " obj_Name" V_SDA;
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Element

- obj_name

The net that can be wired in the internal logic is used as obj_name

Examples

```
NET_LOC " netname0" V_SCL;
```

```
NET_LOC " netname1" V_SDA;
```

//The netname0 is used to control SCL functions, and The netname1 is used to control SDA functions.

Note!

I2C net constraints are not supported for the following devices: GW5A(S)(T)-138 and GW5AT-75.

2.10.8 NET_NO_REP Constraints

Specifies a constraint that prevents resource replication for high-fanout nets.

Syntax

```
NET_NO_REP "netName";
```

Note!

For syntax rules, you can see [Syntax Rules](#) in Section 2.1.

Constraint Elements

- netName

A high-fanout net used as netName. Multiple netName values can be specified at once.

Examples

Example 1

```
NET_NO_REP "net1";
```

// Specifies the high-fanout net (net1), which will not use resource replication for fanout optimization.

Example 2

```
NET_NO_REP "net2","net3";
```

// Specifies the high-fanout nets (net2 and net3), which will not use resource replication for fanout optimization.

3 FloorPlanner

3.1 Introduction

FloorPlanner is a physical constraint editor and designed in-house by Gowin. It supports reading and editing the attributes and locations of I/O, Primitive, and Group, etc. It also supports the generation of place constraints files according to your configuration. These files define I/O attributes, as well as locations of primitives and modules. FloorPlanner provides easy and fast placement and constraint editing functions to improve the efficiency of writing physical constraint files.

The functions of FloorPlanner are as follows.

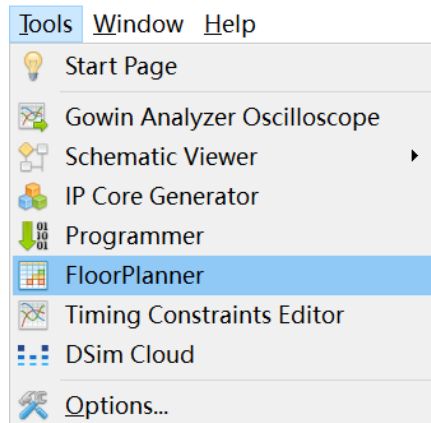
- Supports the input of user design files & constraint files, editing constraint files, and the output of constraint files
- Supports the display of I/O Port, Primitive and Group constraints in user design files
- Can create, edit and modify constraints files
- Supports grid mode, macro cell mode and primitive mode of chip array
- Supports Package View
- Can display Chip Array and Package View synchronously
- Supports real-time and differential display of constraint locations
- Can generate locations by dragging
- Supports I/O port attribute configuration and batch configuration
- Supports display and editing functions of Clock Net Constraints
- Supports constraints legality check
- Supports Back-annotate Physical Constraints

3.2 Start FloorPlanner

There are three methods to start FloorPlanner:

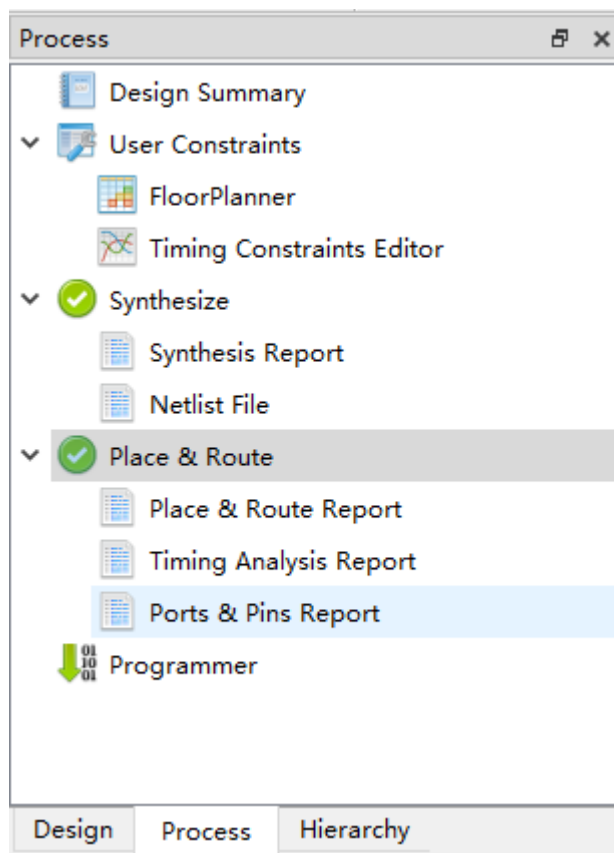
1. Click "IDE > Tools" to open "FloorPlanner", as shown in Figure 3-1.

Figure 3-1 Start FloorPlanner via Menu Bar



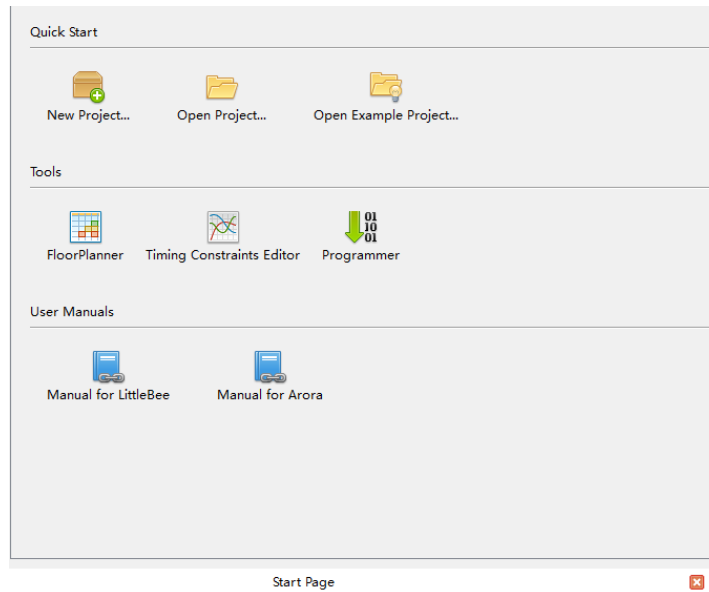
2. After synthesis, double-click "FloorPlanner" in Process view, as shown in Figure 3-2.

Figure 3-2 Start FloorPlanner in Process View



3. Click "IDE > Start Page>Tools> FloorPlanner" to open FloorPlanner, as shown in Figure 3-3.

Figure 3-3 Start FloorPlanner via Start Page



Note!

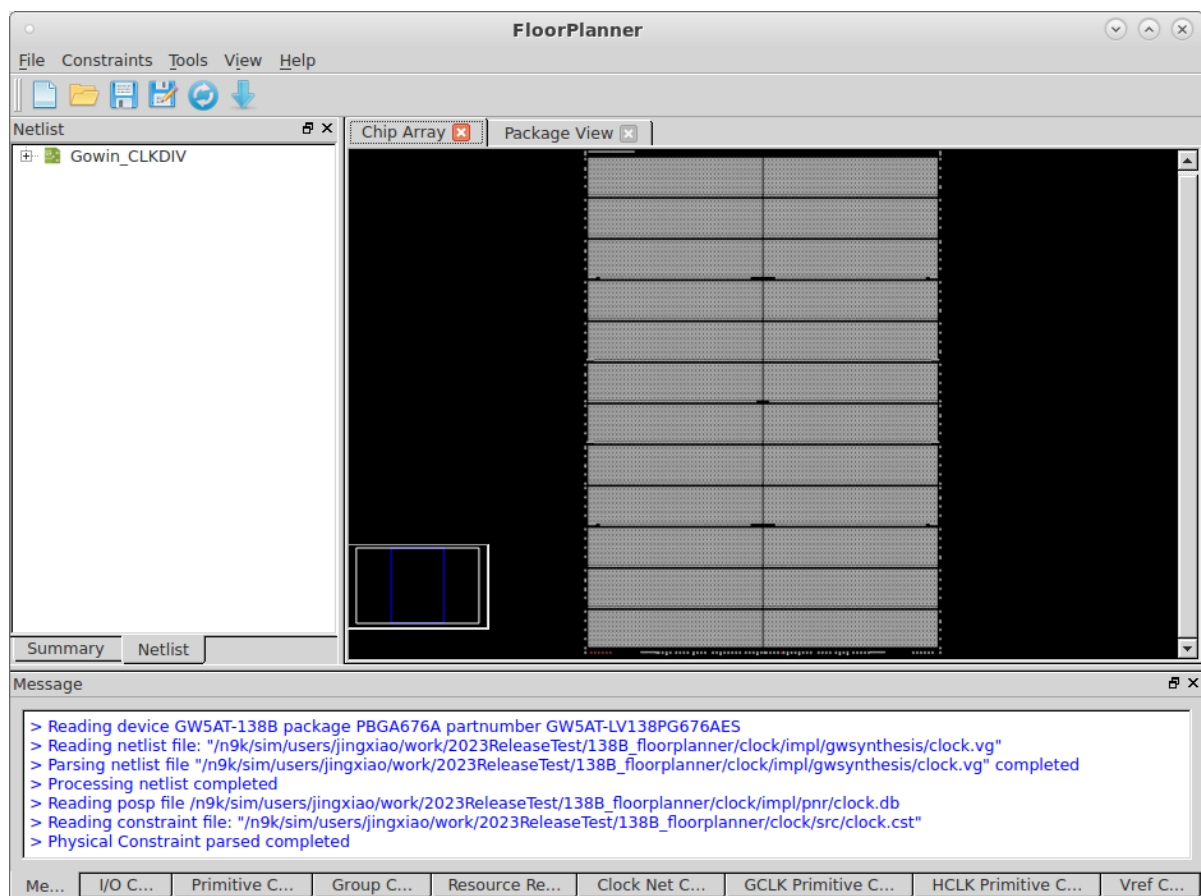
- If you use Gowin FloorPlanner for constraints, the netlist file should be added first.
- When you choose the first or the second method to start Gowin FloorPlanner, the netlist file will be loaded automatically.
- When you choose the third method to start Gowin FloorPlanner, the netlist file is needed to be loaded via "File > New" option.
- When starting FloorPlanner using the first or second method, the interface layout from the last time FloorPlanner was closed will be remembered for the current project.

3.3 FloorPlanner Interface

Create or open FloorPlanner interface (including the netlist file), as shown in Figure 3-4.

The interface displays menu, toolbar, Netlist, Summary, Chip Array, Package View, and Message, etc.

Figure 3-4 FloorPlanner Interface



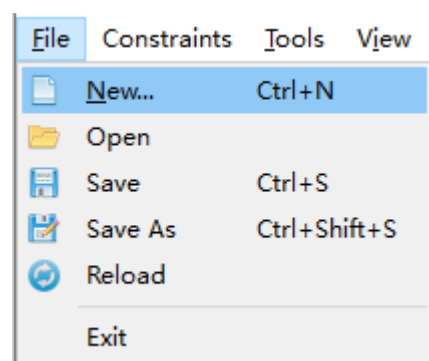
3.3.1 Menu Bar

The menu bar includes "File", "Constraints", "Tools", "View", and "Help".

File Menu

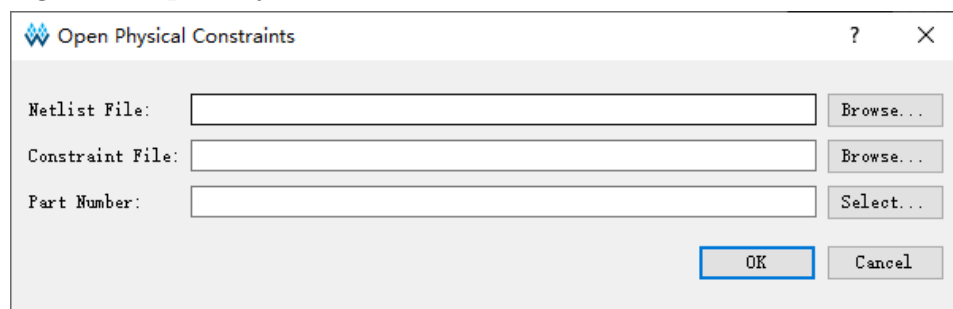
File menu is shown in Figure 3-5.

Figure 3-5 File



- **New:** Create constraints, add user design and select device, etc.
- **Open:** Open to add constraints, select part number, as shown in Figure 3-6.
- **Reload:** Physical constraint files, and place files can be reloaded after modifying.
- **Save:** Save the modified files.
- **Save As:** Save the modified file to a specified file and use the netlist name as the name of constraint file, which can be modified.
- **Exit:** Exit FloorPlanner.

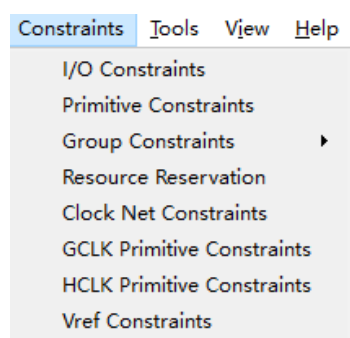
Figure 3-6 Open Physical Constraints



Constraints Menu

Constraints menu is as shown in Figure 3-7.

Figure 3-7 Constraints



Primitive Constraints

Right-click to select "Select Primitives" and a dialog box pops up, as shown in Figure 3-8.

You can select primitives by name or type.

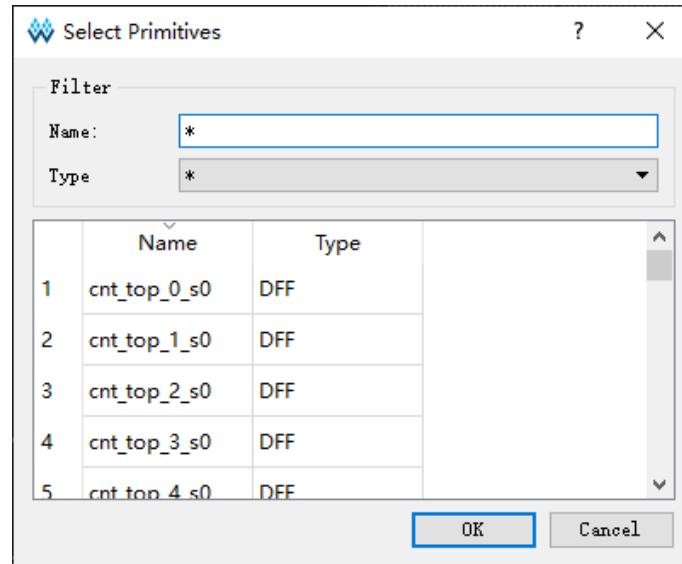
1. Click "OK" to generate the constraints and the constraints are displayed in "Primitive Constraints" at the bottom of main interface.

2. You can set the location by typing or dragging in editing view.

Note!

The location is highlighted in light blue in Chip Array.

Figure 3-8 Select Primitive



Group Constraints

Group constraints include New Primitive Group and New Relative Group.

Create Primitive Group

1. Create primitive group. Right-click to select "New Primitive Group" and a dialog box pops up, as shown in Figure 3-9.
2. You can set Group name, Primitives locations and Exclusive. you can add and remove Primitives by clicking "+" and "-" buttons to create a right Primitive Group, as shown in Figure 3-10.

Note!

- Group name, Primitive, and Locations are required.
- Locations can be inputted in the following ways:
 - Manually input
 - Before creating group constraints, copy the location and paste it into "New Primitive Group > Locations" in Chip Array window.
- 3. After finishing configuration, click "OK", and the syntax of the locations will be checked by the tool.
 - If the location is invalid, a prompt dialog box as Figure 3-11 and Figure 3-12 will pop up. You need to change the location.

- If there is no error, click "OK", and the available location will be displayed in Chip Array.
4. You can see the created group constraints in "Group Constraints". Double-click the group constraint, and Figure 3-10 pops up; you can edit the constraints.

Figure 3-9 New Primitive Group

Group Name:

Members

Name	Type
------	------

☐ Exclusive

Locations

☐ Exclusive

Figure 3-10 Right Primitive Group

Group Name:

Members

Name	Type
cout_cZ	LUT4
out_Z[1]	DFFE

☐ Exclusive

Locations

☐ Exclusive

Figure 3-11 Invalid Locations

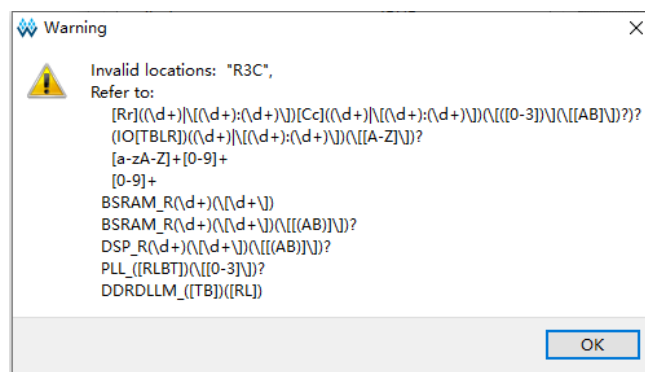
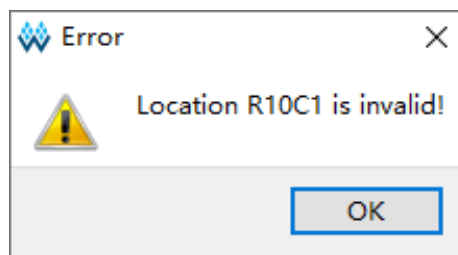


Figure 3-12 Invalid Locations



Create Relative Group

1. Create Relative Group constraints. Right-click to select "New Relative Group" and a dialog box pops up, as shown in Figure 3-13.
2. You can set the group name, group primitives, and their relative locations. You can add and remove primitives by "+" and "X" buttons. The created relative group constraints are shown in Figure 3-14.

Note!

- Group name, Primitive, and Relative Location are required.
 - The locations can be inputted in the following ways:
 - Manually input
 - Before creating group constraints, copy the location and paste it to "New Relative Group > Relative Location" in Chip Array window.
3. Click "OK" to generate the constraints.
 4. See the created constraints in "Group Constraints". Double-click the constraints, and a dialog box pops up, as shown in Figure 3-14; you can edit the constraints.

Figure 3-13 New Relative Group

Group Name:

Members

Primitive	Relative Location
-----------	-------------------

Figure 3-14 Right Relative Group

Group Name:

Members

Primitive	Relative Location
cout_4_cZ	R5C3
outlde	R7C2

Resource Reservation

1. Create Resource Reservation constraints. Click Reserve Resources to create a new constraint in "Resource Reservation" window at the bottom of the interface.
2. You can set the locations by typing or dragging.
3. Double-click "Attribute" or click the "Attribute" column drop-down box to set the attribute of the reserved locations, as shown in Figure 3-15.

Note!

Name is used to distinguish reserved constraints. The name cannot be modified.

Figure 3-15 Resource Reservation

	Name	Locations	Attribute
1	reserve_0	drag or type t...	ALL
			ALL
			LUT
			REG

Clock Net Constraints

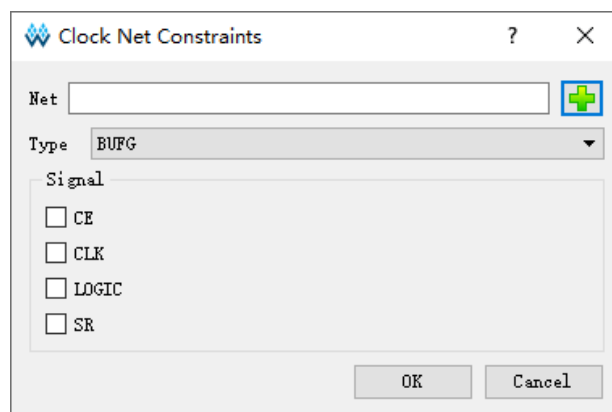
Create global clock constraints and the number of constraints is limited; and the constraints validity will be checked. Right-click to select "Clock Net Constraints" and a dialog box pops up, as shown in Figure 3-16. You can perform the following operations.

1. Click " " to select the corresponding Net.
2. Select "BUFG", "BUFG[0]~[15]", and "LOCAL_CLOCK" in "Type" drop-down list.
3. Configure Signal by "CE" and "CLK" check box. After finished, click "OK" to generate constraints in "Clock Net Constraints". Double-click to open the dialog box for editing.

Note!

When LOCAL_CLOCK selected, the signal check box is grayed.

Figure 3-16 Clock Assignment



GCLK Primitive Constraints

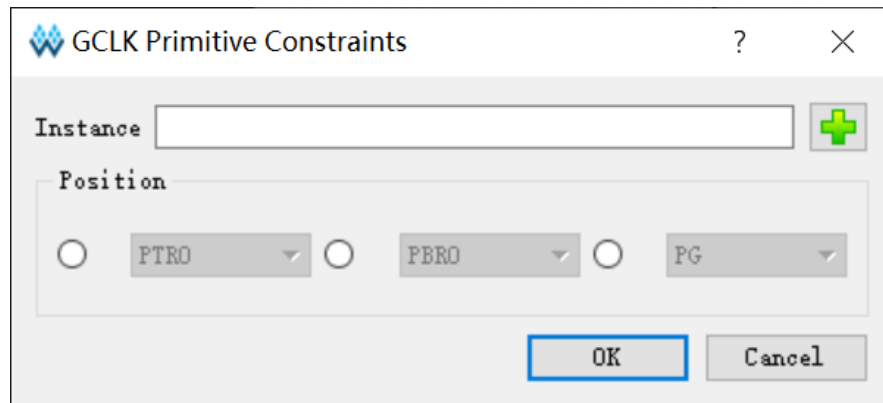
GCLK Primitive Constraints is used to create global clock constraints for the global clock primitives, which constrains the specified Instance to a specific global clock according to the global clock distribution of the device; use GW5AT-138 device as an example, right-click on the "GCLK Primitive Constraints" constraint editing window at the bottom of the main interface and select the "GCLK Primitive Constraints", the dialog box shown in Figure 3-17 pops up. The related operations are shown as follows:

1. Select the corresponding GCLK primitive by clicking the " " button, if there is no GCLK primitive in the design, you cannot add.
2. Configure the global clock position using "Position" and the drop-down

list.

- Click "OK" to generate the constraints, which are displayed in the "GCLK Primitive Constraints"; double click the editing window to reopen the constraint dialog box, and you can edit and modify the constraints.

Figure 3-17 GCLK Primitive Constraints



Note!

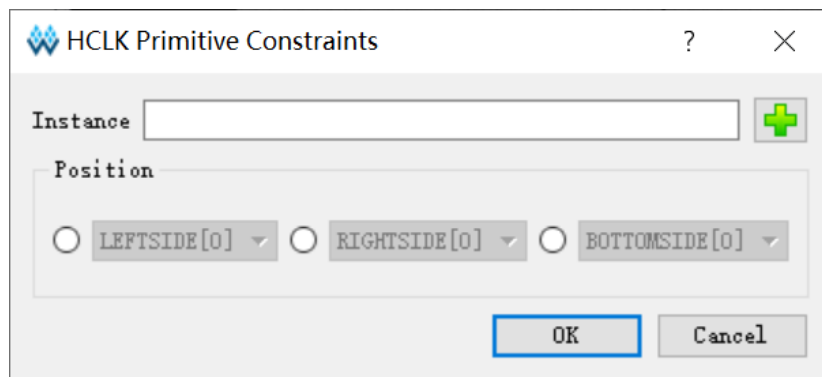
- When "Instance" is selected, "Position" is highlighted.
- Position varies depending on the device, and the position available for different global clock primitives will also vary.

HCLK Primitive Constraints

HCLK Primitive Constraints is used to create constraints for the HCLK primitives, and specify the constraints to high-speed clock locations; use GW5AT-138 device as an example, right-click on the "HCLK Primitive Constraints" constraint editing window at the bottom of the main interface and select "Select HCLK Primitive", the dialog box shown in Figure 3-18 pops up. The related operations are shown as follows:

- Select the corresponding HCLK primitive by clicking the "+" button, if there is no GCLK primitive in the design, you cannot add.
- Configure the high-speed clock position using "Position" and the drop-down list.
- Click "OK" to generate the constraints, which are displayed in the "HCLK Primitive Constraints "; double click the editing window to reopen the constraint dialog box, and you can edit and modify the constraints.

Figure 3-18 HCLK Primitive Constraints

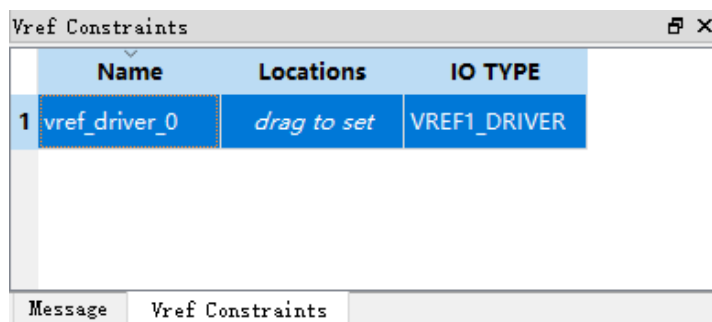
**Note!**

- When "Instance" is selected, "Position" is highlighted.
- Position varies depending on the device, and the position available for different global clock primitives will also vary.

Vref Constraints

Create Vref Driver to configure IO Port Vref; right-click and select "Define Vref Driver" to create a new constraint in the "Vref Constraints" window, as shown in Figure 3-19.

Figure 3-19 Vref Constraints

**Note!**

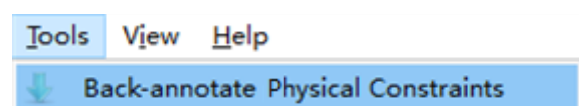
- Specify the location of Vref by dragging.
- Modify Vref name by double-clicking.

Tools Menu

Tools menu is shown in Figure 3-20.

Back-annotate Physical Constraints: Back annotate each primitive and I/O place information to physical constraints file.

Figure 3-20 Tools



1. Click "Tools > Back-annotate Physical Constraints" to open a dialog box, as shown in Figure 3-21. Back-Annotate Physical Constraints is effective only when FloorPlanner is started in the project after Place & Route runs successfully.
2. You can select one or more objects in the Back-annotate Physical Constraints dialog box. Click "OK" to open the "Save as" dialog box and print the place information to the physical constraint file.
3. As shown in Figure 3-22, it is the generated physical constraints file when "Port" and "Port Attribute" selected in Back-annotate Physical Constraints.

Figure 3-21 Back-annotate Physical Constraints

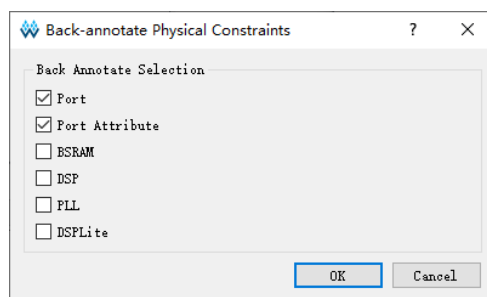


Figure 3-22 Back-annotate Port

```

1
2 IO_LOC "in1" N13;
3 IO_PORT "in1" IO_TYPE=LVCOS33 PULL_MODE=NONE BANK_VCCIO=3.3;
4 IO_LOC "in2" G18;
5 IO_PORT "in2" IO_TYPE=LVCOS33 PULL_MODE=NONE BANK_VCCIO=3.3;
6 IO_LOC "in3" H14;
7 IO_PORT "in3" IO_TYPE=LVCOS33 PULL_MODE=NONE BANK_VCCIO=3.3;
8 IO_LOC "in4" J16;
9 IO_PORT "in4" IO_TYPE=LVCOS33 PULL_MODE=NONE BANK_VCCIO=3.3;
10 IO_LOC "in5" J15;
11 IO_PORT "in5" IO_TYPE=LVCOS33 PULL_MODE=NONE BANK_VCCIO=3.3;

```

View Menu

As shown in Figure 3-23, View includes Toolbars, Windows, Zoom In, Zoom Out and Zoom Fit. The descriptions of these sub-menus are as follows.

- Reset Layout: Restore settings of the initial interface layout.
- Toolbars: Display shortcuts
- Windows: Display different windows, as shown in Figure 3-24
- Zoom In: Zoom in Chip Array or Package View
- Zoom Out: Zoom out Chip Array or Package View

- Zoom Fit: Zoom in/out Chip Array or Package View according to the window size.

Figure 3-23 View Menu

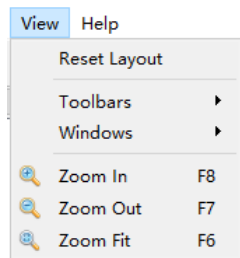
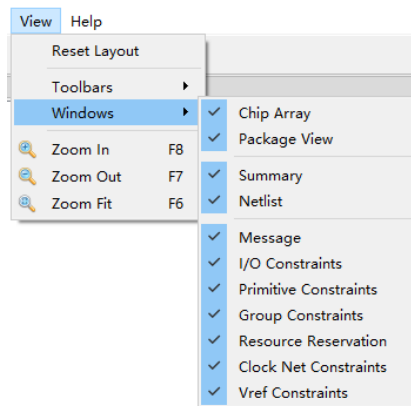


Figure 3-24 Windows Menu



Help Menu

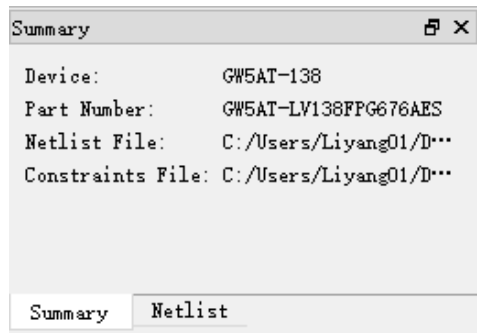
Help is used to provide software version and copyright information.

3.3.2 Summary and Netlist Windows

Summary and Netlist windows display device, part number, user design, constraints path and netlist, etc.

Summary Window

The summary window is shown in Figure 3-25. It displays the information of device, part number, design files and constraints files.

Figure 3-25 Summary Window

Netlist Window

As shown in Figure 3-26, Netlist window displays Ports, Primitives, Nets, and Module.

Note!

- The Ports and Primitives names are displayed in full path and sorted in alphabetical ascending order by default.
- Port and Net display via Bus and non-Bus, as shown in Figure 3-27.
- Modules are displayed in hierarchy and the number of instances in each module is also displayed, as shown in Figure 3-28.

Figure 3-26 Netlist View

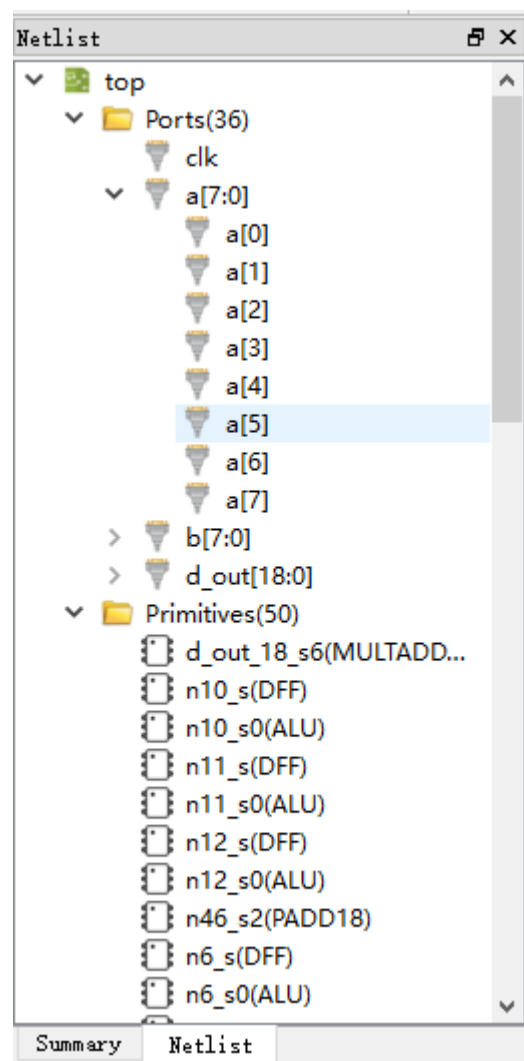


Figure 3-27 BUS and Non-Bus Display

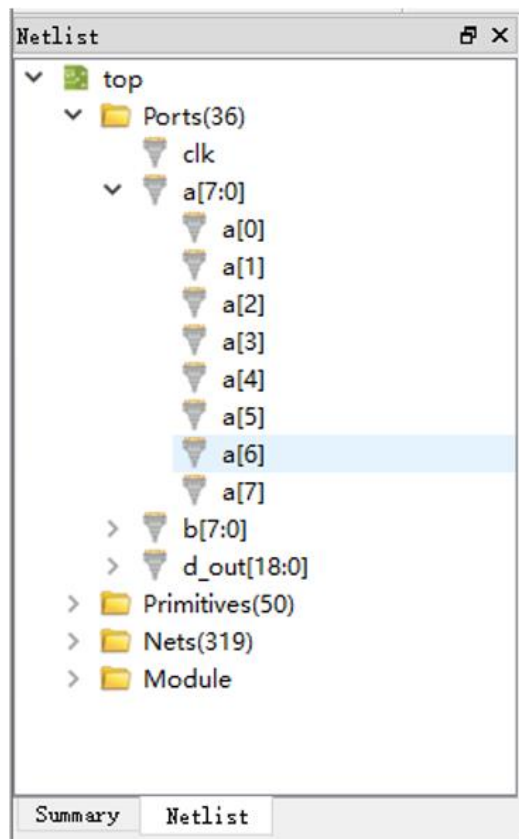
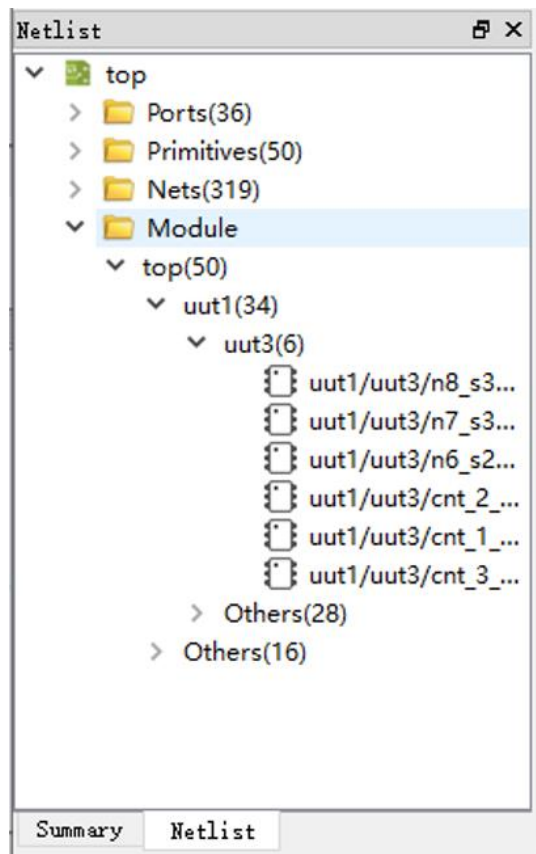


Figure 3-28 Hierarchy Display



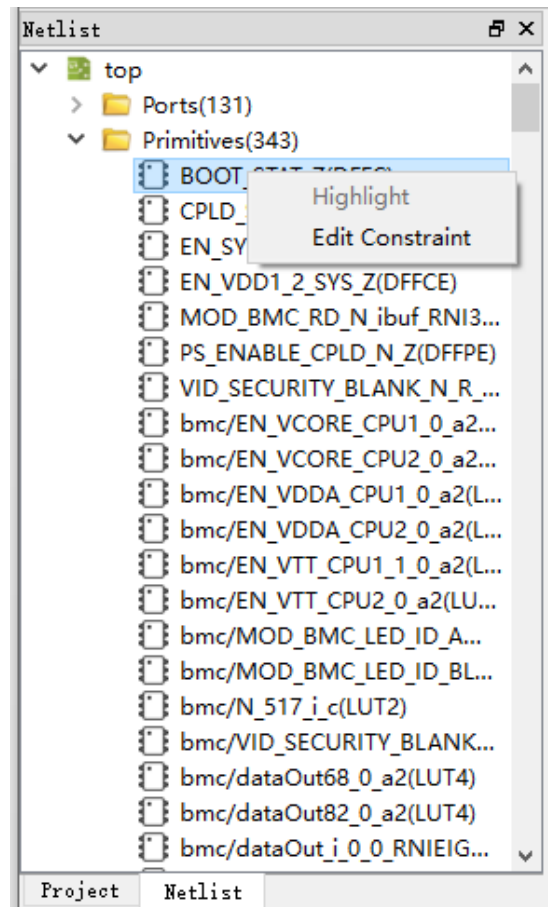
Netlist provides right-click menu as shown below.

- Highlight: Highlight the corresponding constraint location in Chip Array.
- Edit Constraint: Edit the corresponding constraints.

Note!

If the current Primitive or Port has no constraints locations, the highlight is not available, as shown in Figure 3-29.

Figure 3-29 Netlist Right-clicking



3.3.3 Package View

As shown in Figure 3-30 , taking GW5AT-138-FCPBGA676A as an example, Package View displays I/O, power, and ground pins based on chip package. When the mouse is placed on a location, the I/O type, bank, and LVDS will display.

Figure 3-30 Package View (GW5AT-138-FPBGA676A)



Various symbols and colors are used to distinguish user I/Os, power supply pins and ground pin. The colors of IO pins of different BANKS are different, as shown below.

- "⚡": User I/O
- "⚡": VCCIO
- "⚡": VSS

The right-click menu supported by the Package View is shown in Figure 3-31. The descriptions are as follows.

- Zoom In: Zoom in Package View
- Zoom Out: Zoom out Package View
- Zoom Fit: Zoom in/out Package View to fit window size
- Show Differential IO Pairs: Display differential pair. As shown in Figure 3-32, a differential pair is connected by a red line.
- Top View: Package View is displayed in the top view by default. The

top view of GW5AT-138-FCPBGA676A with coordinate origin at the top left corner is as shown in Figure 3-33.

- Bottom View: The bottom view of GW5AT-138-FCPBGA676 with coordinate origin at the top right corner is as shown in Figure 3-34.

Figure 3-31 Package View Right-clicking

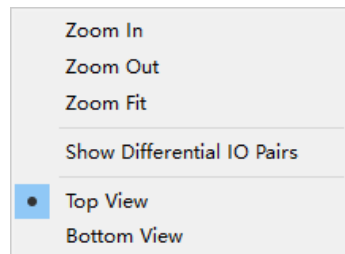


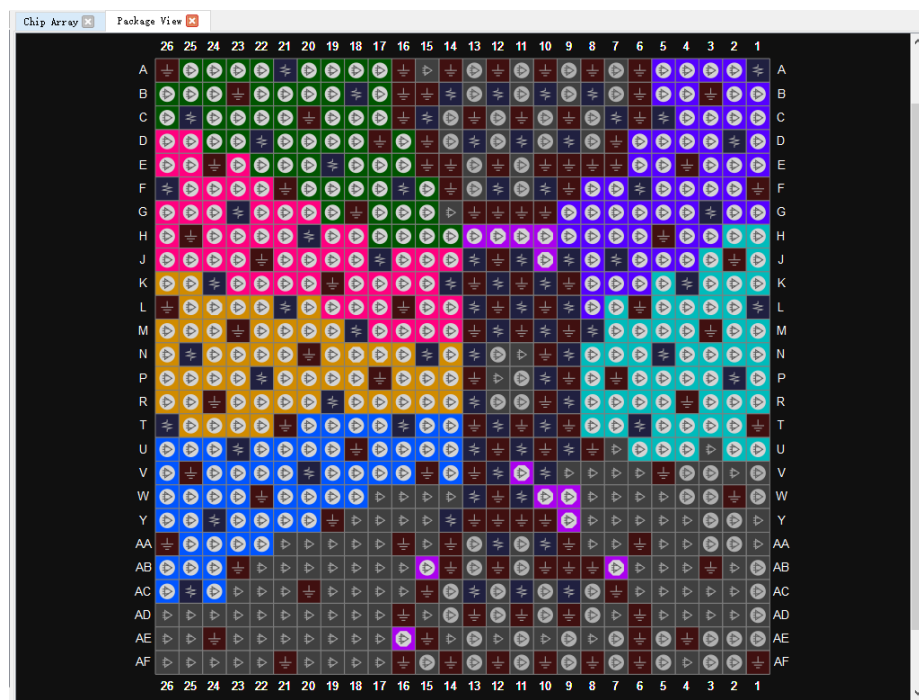
Figure 3-32 Differential Pair Display



Figure 3-33 Top View



Figure 3-34 Bottom View



Package View supports the display of IO Port constraint locations, and the IO Port can be constrained by dragging from the Netlist or I/O Constraints to the Package View window. When dragged, the port name will be displayed; the unconstrained pins are grayed out and not dragged.

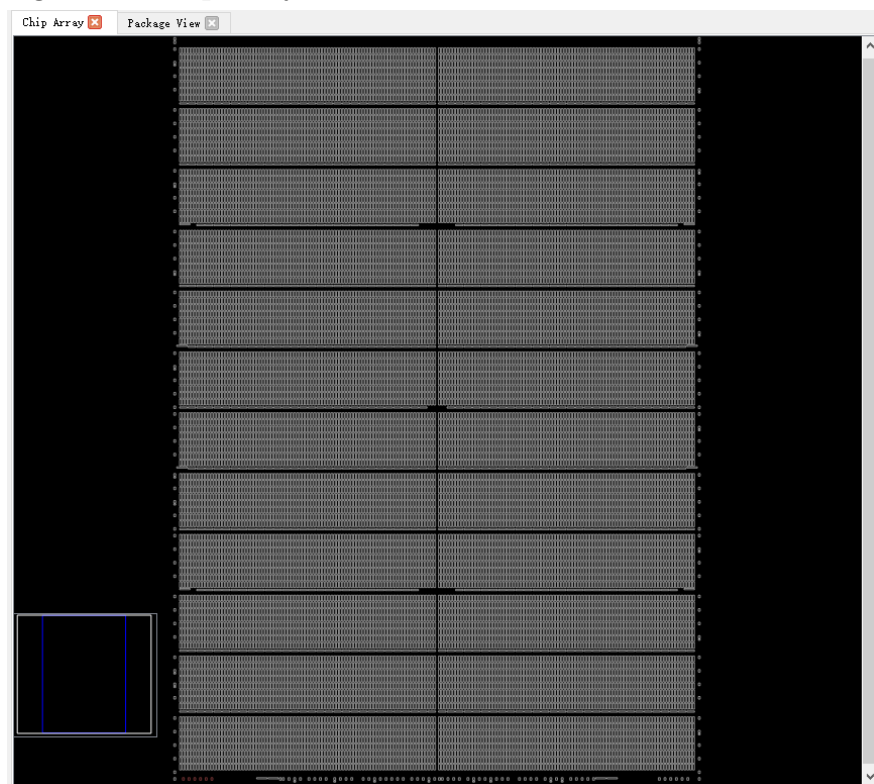
3.3.4 Chip Array Window

The Chip Array window of FloorPlanner is shown in Figure 3-35. Chip Array displays I/O, CFU, CLU, DSP, PLL, BSRAM, and DQS according to the row and column information of the chip, supports real-time display of all constraints locations, and also supports the functions of zoom in/out and dragging, etc.

I/O denotes all I/O locations of die and is distinguished with different colors.

- White: Bonded I/O location
- Red: Unbonded I/O location

Figure 3-35 Chip Array Window



Chip Array provides grid mode, macrocell mode, and primitive mode.

- Grid mode: Display constraint locations in grid, as shown in Figure 3-36.
- Macrocell mode: Display constraint locations in CLS, IOBLK, as shown in Figure 3-37.
- Primitive mode: Display constraint locations in REG, LUT etc., as shown in Figure 3-38.

Figure 3-36 Constraints in Grid

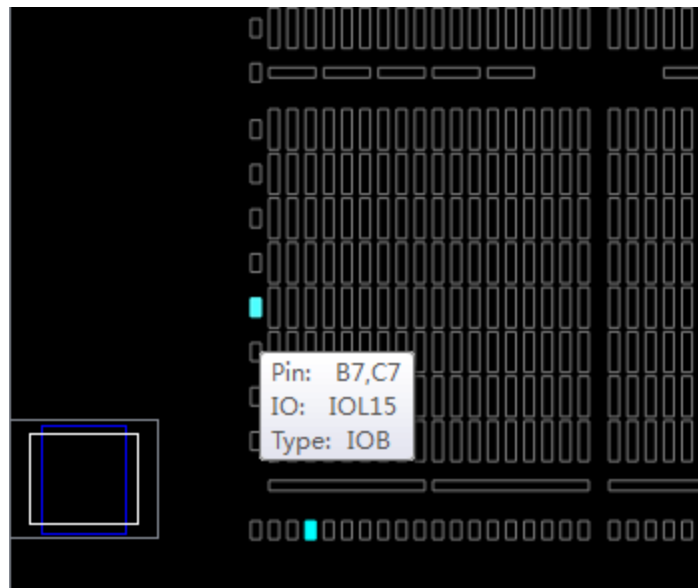


Figure 3-37 Constraints in Macrocell

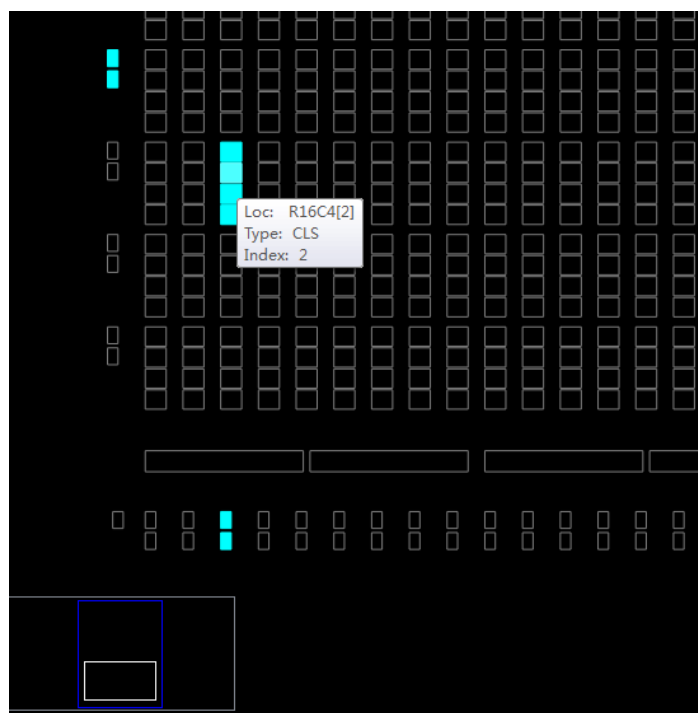
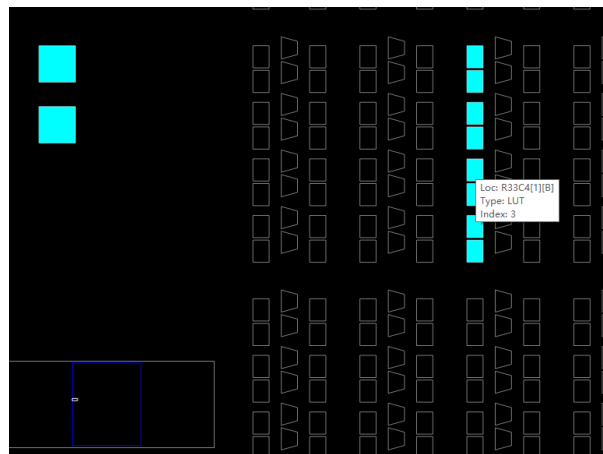


Figure 3-38 Constraints in Primitive



Chip Array supports following dragging functions.

- Drag from Netlist to Array to generate constraints and specify constraint location.
- Drag from Editing to Array to specify constraint location.

There is a chip sub-window in Chip Array for real-time display of the current view relative to the device. Drag the white frame in the chip sub-window to move Chip Array. Chip Array distinguishes constraints type and displays constraints locations in different colors. The color meaning is as follows.

- White: Display constraints location being selected or highlighted.
- Dark blue: Display reserved constraints, indicating that the location cannot be occupied again.
- Light blue: Display IO and Primitive constrained in a grid or range.

Chip Array supports right-clicking functions are as follows.

- Zoom In: Zoom in Chip Array
- Zoom Out: Zoom out Chip Array
- Zoom Fit: Zoom in/out Chip Array to fit the window size
- Show Constraints View: Show instances constraints view of Chip Array
- Show Place View: Show instances place view of Chip Array; it is only effective when FloorPlanner is started after running Place & Route. Otherwise, it is greyed out.
- Show Multi-View: Show instances constraints and place views of Chip Array; it is only effective when FloorPlanner is started after running

Place & Route. Otherwise, it is greyed out.

- Show In-Out Connection: Show and select the input and output connection of the instance in the Place View; it can only be used if an instance is selected when Show Place View > All Instance view opens. Otherwise, it is greyed out.
- Show In Connection: Show and select the input connection of the instance in the Place View; it can only be used if an instance is selected when Show Place View > All Instance view opens. Otherwise, it is greyed out.
- Show Out Connection: Show and select the output connection of the instance in the Place View; it can only be used if an instance is selected when Show Place View > All Instance view opens. Otherwise, it is greyed out.
- Unhighlight All: Remove highlight.
- Copy Location: Copy the selected location or area; If GRID and Block are selected, "Copy Location" in right-click menu is available. Otherwise, it is unavailable, as shown in Figure 3-39.

Show Place View also shows Lut and Reg density, as shown in Figure 3-40.

- ALL Instance: Show place of all instances. Light green indicates less than five, green indicates six to ten, and dark green indicates more than ten.
- All Lut: Shows place of all Lut. Light green indicates less than two, green indicates three to four, and dark green indicates more than four.
- All Dff: Shows place of all Reg. Light green indicates less than two, green indicates three to four, and dark green indicates more than four.

You can view the place of all instances in the design by clicking Show Place View > ALL Instance.

- Hover the mouse over the instance place location in the Chip Array window to display the name of the instance, as shown in Figure 3-41.
- Select a specific instance in the Netlist window; right-click and select "Highlight", the place location of that instance will be highlighted, as shown in Figure 3-42.

Note!

You can select an area by "Ctrl" + left mouse button dragging; right click and select "Copy Location", you can copy the location and paste it to any constraint editing window.

Figure 3-39 Chip Array Right-clicking

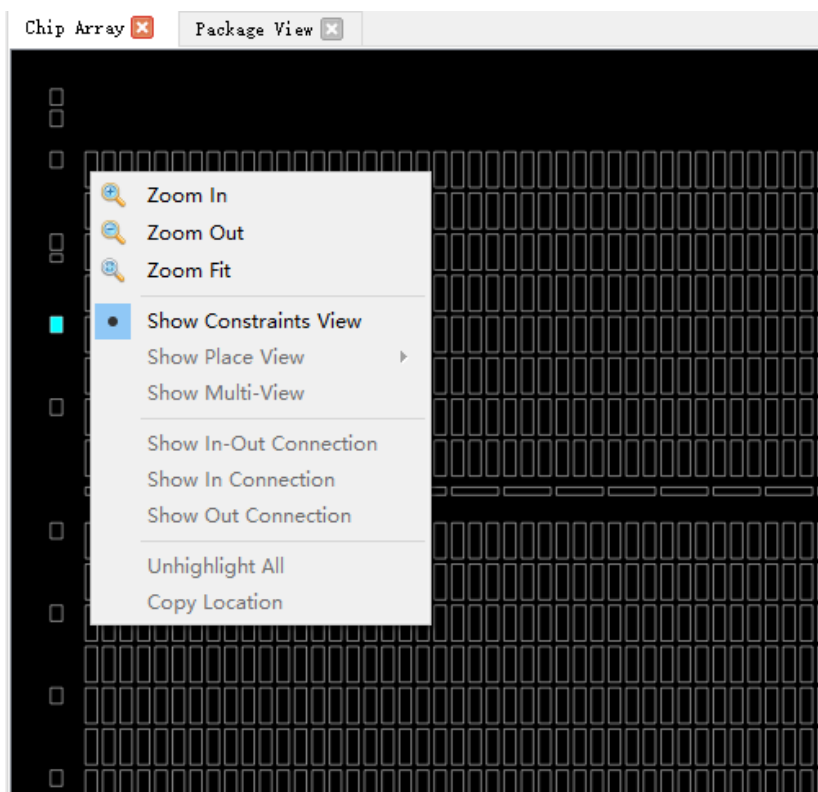


Figure 3-40 Show Place View

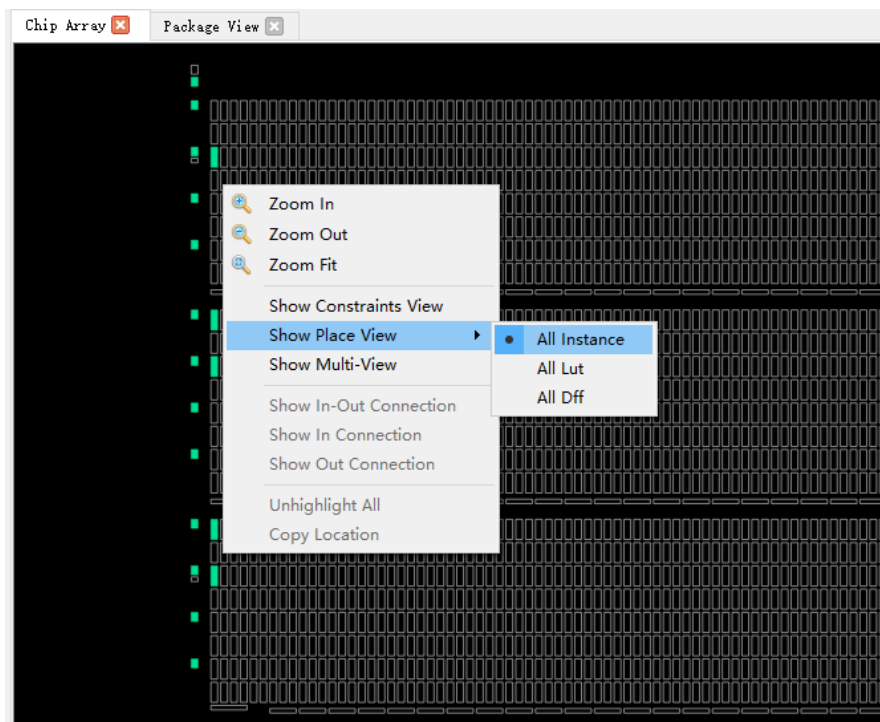


Figure 3-41 Mouse Hovering Display

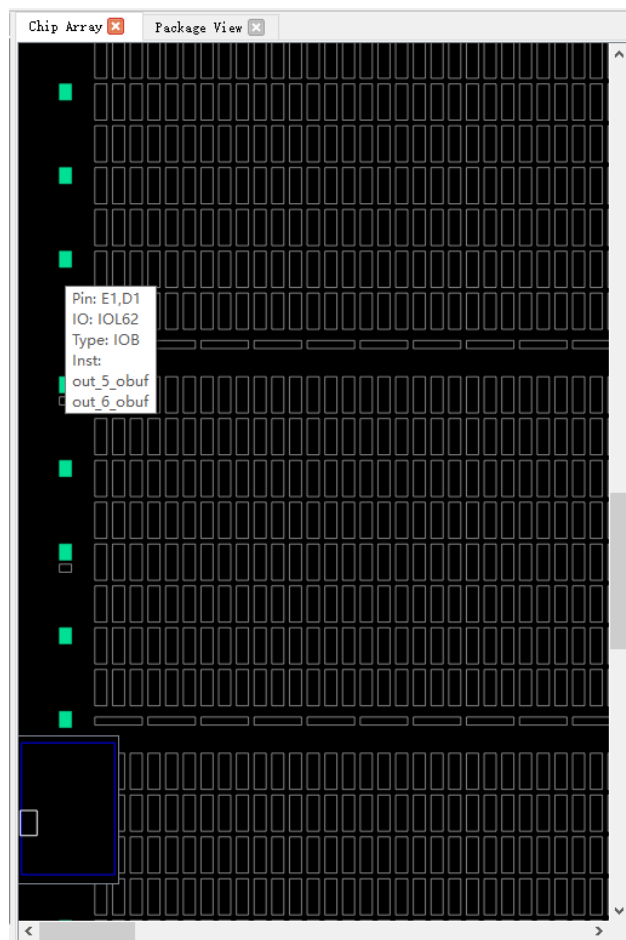
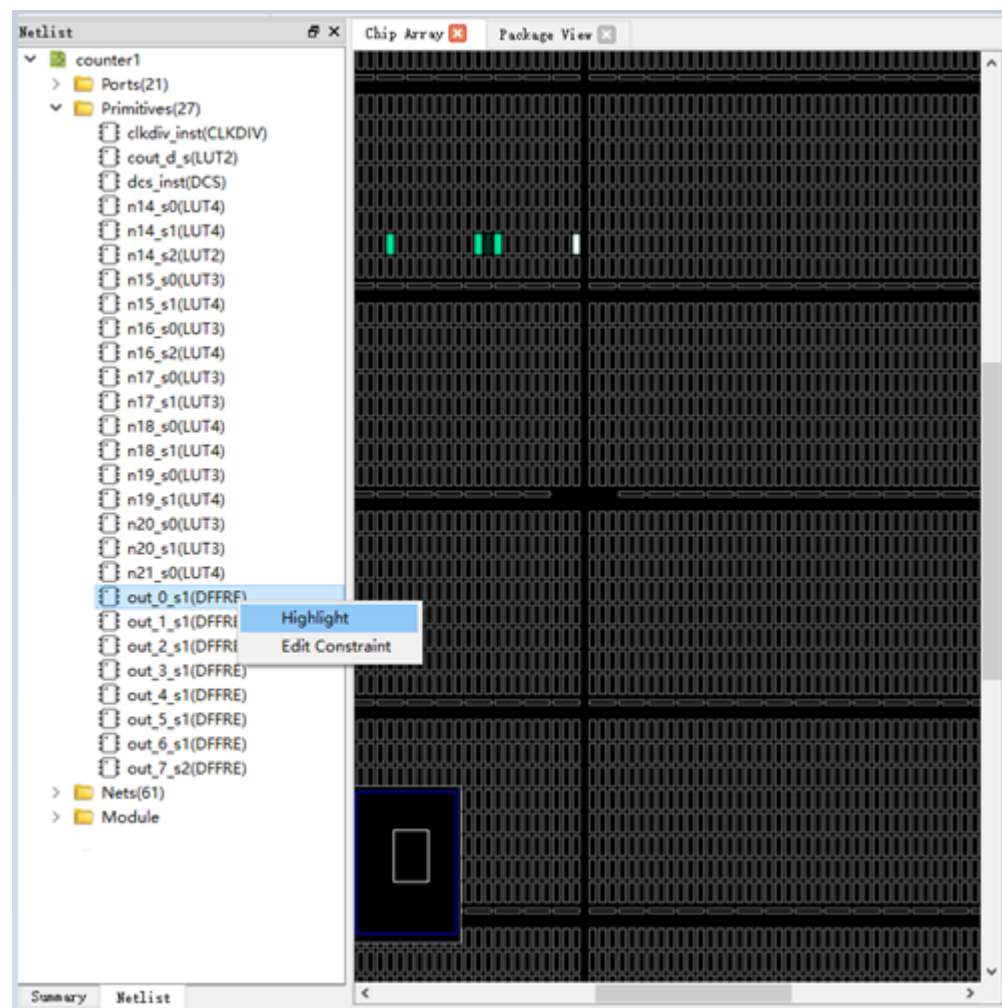


Figure 3-42 Right-click to Select Highlight



3.3.5 Constraints Editing Window

Constraints editing window includes eight constraints views, such as, "I/O Constraints", "Primitive Constraints", "Group" Constraints, etc., which are used to display constraints and provide constraints editor and drag function. The brief introduction of each view is as follows.

I/O Constraints

I/O Constraints is used to constrain ports. I/O constraint view is as shown in Figure 3-43, and the functions are as follows.

- Display IO Port attributes and constraints in user design, such as Direction, Bank, IO Type, Pull Mode, etc.
- Support to edit constraints locations and attribute, etc.
- Change constraints by dragging, double-clicking, etc.

Note!

- Set I/O location by dragging or double-clicking.
- Display IO name when dragged.
- When I/O is dragged into Chip Array window, the location where I/O can be placed is brightened, and the color of the location where I/O cannot be placed remains unchanged.
- When I/O is dragged into Package View window, the color of the location where I/O can be placed remains unchanged and the location where I/O cannot be placed becomes darker.
- After setting, the constraints location in Chip Array is highlighted in light blue, and the constraints location in Package View is highlighted in orange.

The details of the right-click menu are as follows.

- Unplace: Cancel placement
- Reset Properties: Reset Port properties
- Highlight: Highlight constraints location
- IO Type: Set the level standard
- Drive: Set drive current
- Pull Mode: Set pull-up mode
- Pull Strength: Set pull-up strength
- PCI Clamp: Set the switch of PCI protocol
- Hysteresis: Set hysteresis
- Open Drain: Turn on/off open drain
- Vref: Set reference voltage
- Single Resistor: Turn on/off single-ended resistor
- Diff Resistor: Turn on/off differential resistor
- Bank Vccio: Set BANK voltage
- Ctle: Continuous Time Linear Equalizer, used to compensate for channel attenuation
- Slew Rate: Used to set the voltage slew rate

Note!

- You can modify port attributes in batches by right-clicking; if you select multiple ports, and these ports have the same attribute values to be configured, they can be configured in batches. For the details, you can see [DS981, Arora V 138K & 75K FPGA Products Data Sheet](#).
- GW5A(T)-60 and GW5A(N)(R)T-15 devices support Slew Rate setting.

Figure 3-43 I/O Constraints View

	Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type	Drive	Pull Mode	PCI Clamp	Hys
2	cin	input				False	LVC MOS18	N/A	UP	N/A	N
3	clk	input				False	LVC MOS18	N/A	UP	N/A	N
4	clko	output				False	LVC MOS18	8	UP	N/A	I
5	cout	output				False	LVC MOS18	8	UP	N/A	I
6	data[0]	input				False	LVC MOS18	N/A	UP	N/A	N
7	data[1]	input				False	LVC MOS18	N/A	UP	N/A	N

Primitive Constraints

Primitive Constraint is used to constrain primitive location, as shown in Figure 3-44, and the functions are as follows:

- Display the name, type, location, and Exclusive of all Primitive constraints;
- Support editing; you can highlight, remove, add and update constraints by right-clicking.

Note!

- Modify the locations by dragging or double-clicking
- Set Exclusive by double-clicking
- Syntax and legality will be checked for the locations when manually writing primitive constraints, and error message dialog boxes are as shown in Figure 3-11 and Figure 3-12.

Figure 3-44 Primitive Constraints View

	Primitive	Type	Locations	Exclusive
1	ALU_check	ALU	P4G12	False

Group Constraints

Group Constraints is used for group constraints on the I/O and some primitives in the design, the group constraint view is shown in Figure 3-45; and the functions are as follows.

- The view displays the name, type, number of primitive, location, and Exclusive of all group constraints, which includes primitive and relative group constraints. As shown in Figure 3-10 and Figure 3-14, double-

click the group to edit constraints.

- You can highlight, remove, add and update constraints by right-clicking.

Figure 3-45 Group Constraints View

Group	Type	Members Number	Members Exclusive	Locations	Locations Exclusive
1	grp1	Primitive	False	R3C4	False

Resource Reservation

Resource Reservation is used for reservation constraints on the resources available in the current package, as shown in Figure 3-46; and the functions are as follows.

- The view can display reserved constraints locations.
- You can highlight, remove, add and update constraints by right-clicking.
- "Name" is used to distinguish utilization resource of each reservation constraint; and you cannot modify the name.

Note!

You can change the locations by dragging or double-clicking;

Figure 3-46 Resource Reservation View

Name	Locations	Attribute
1	reserve_1	drag or type t...

Clock Net Constraints

Clock Net Constraints is used for global clock assignment constraints on the net in the design, as shown in Figure 3-47, and the functions are as follows.

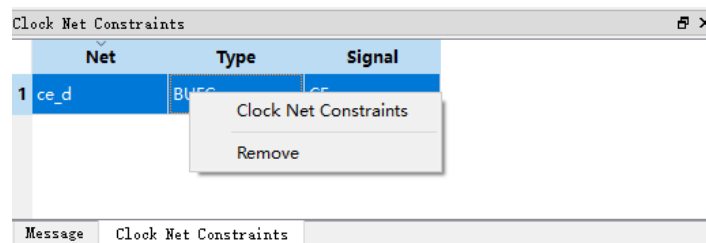
- The view displays all clock assignment constraints.
- You can add and remove clock assignment constraints by right-

clicking.

Note!

- Double-click to edit.
- Dragging is not supported if there is no location.
- Global clock assignment constraint creation is as shown in Figure 3-16.

Figure 3-47 Clock Assignment View



GCLK Primitive Constraints

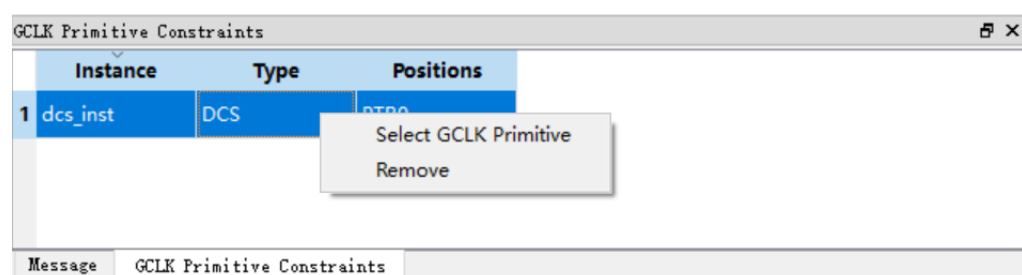
GCLK Primitive Constraints is used to constrain global clock primitives, and its window is as shown in Figure 3-48, with the following functions:

- Display all global clock constraints, including Instance name, type, and quadrant location.
- Support a right-click function for adding new global clock constraints and deleting existing constraints.

Note!

The window for creating a new global clock constraint is shown in Figure 3-17.

Figure 3-48 GCLK Primitive Constraints



HCLK Primitive Constraints

HCLK Primitive Constraints is used to constrain high-speed clock primitives, and its window is as shown in Figure 3-49, with the following functions:

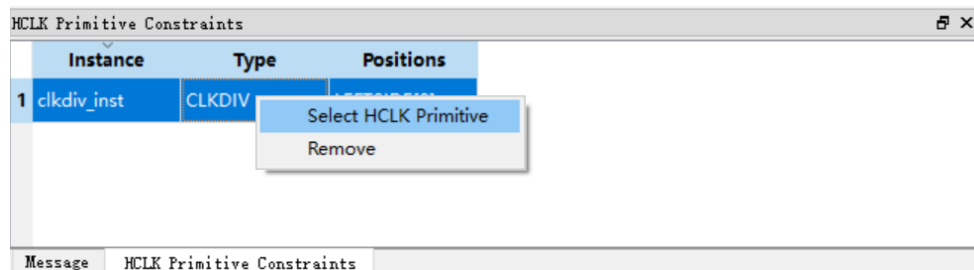
- Display all location constraints for Instance related with high-speed clock, including Instance name, type, and high-speed clock location.

- Support a right-click function for adding new high-speed clock constraints and deleting existing constraints.

Note!

The window for creating a new high-speed clock constraint is shown in Figure 3-18.

Figure 3-49 HCLK Primitive Constraints



Vref Constraints

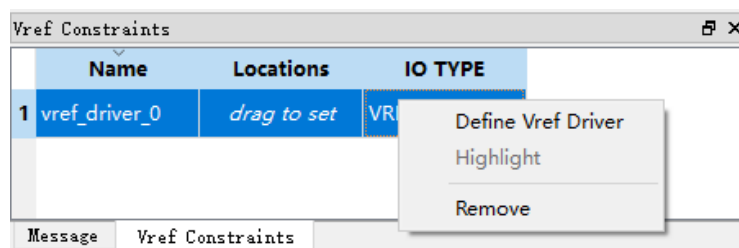
Vref Constraints is used for the external reference voltage of the bank where the constraint is located, as shown in Figure 3-50, and the functions are as follows.

- The view can display Vref Driver defined by users, such as, Vref name and location.
- You can highlight, remove, and add constraints by right-clicking.

Note!

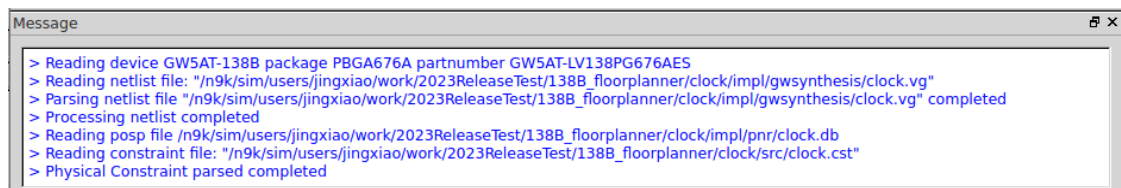
Locations can only be set by dragging.

Figure 3-50 Vref Constraints View



3.3.6 Message Window

The message window is as shown in Figure 3-51, and it displays the output.

Figure 3-51 Message Window

4 Using FloorPlanner

FloorPlanner can create and edit constraints, generate physical constraint files used in Place & Route.

4.1 Create Constraint File

FloorPlanner can output newly created or modified physical constraint files, and the steps are shown below.

1. Start FloorPlanner as described in 3.2 Start FloorPlanner.
2. Click "File > New" to open the "New" dialog box.

Note!

You can also open "New" dialog box in the following two ways.

- Use the "Ctrl+N" shortcut
- Click the "New" icon in the toolbar

3. Select netlist file and part number, as shown in Figure 4-1.

Figure 4-1 New Physical Constraints

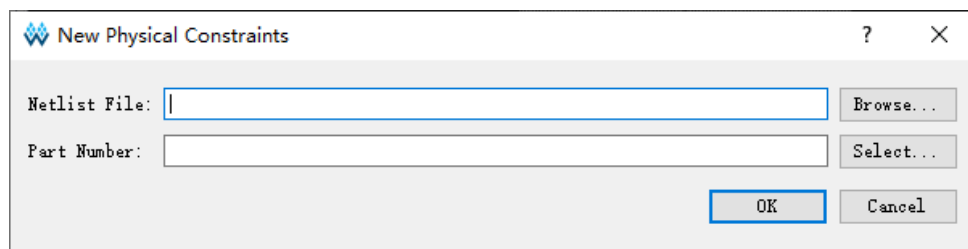


Figure 4-2 Select Device

Select Device

Filter

Series: GW5AT Package: Any

Device: Any Speed: Any

Device Version: Any
*no version number is initial version

Part Number	Device	Device Version	Package	Speed	Voltage	IO	LUT	
GW5AT-LV138PG484AES	GW5AT-138	B	PBGA484A	ES	LV	297	138240	1
GW5AT-LV138GW391AES	GW5AT-138	B	GW391A	ES	LV	324	138240	1
GW5AT-LV138GW391ES	GW5AT-138	B	GW391	ES	LV	324	138240	1
GW5AT-LV138FPG676AES	GW5AT-138	B	FCPBGA676A	ES	LV	312	138240	1
GW5AT-LV138GW391ES	GW5AT-138		GW391	ES	LV	324	138240	1
GW5AT-LV138FPG676AES	GW5AT-138		FCPBGA676A	ES	LV	312	138240	1

OK Cancel

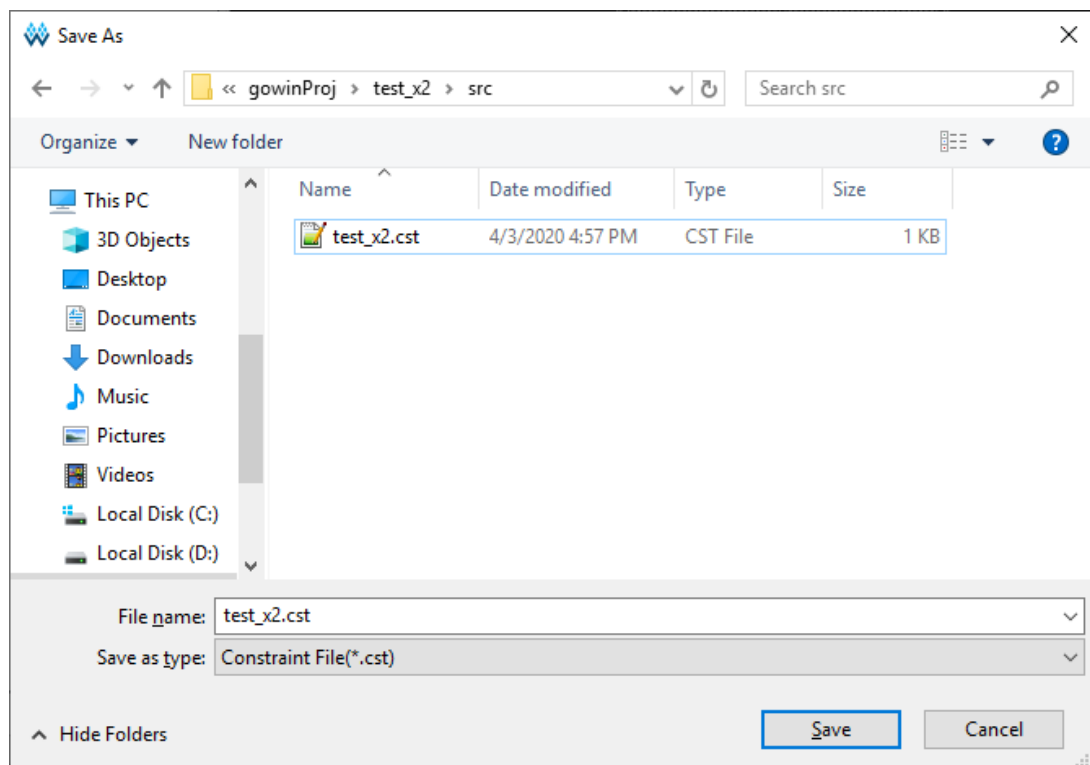
Note!

- You can select the device, package, and all Gowin FPGA devices, as shown in Figure 4-2.
- Start FloorPlanner using the third way in 3.2 Start FloorPlanner.

You can perform the following operations in FloorPlanner:

1. Distribute the pins location by dragging;
2. Click "Save" to output constraint files.
3. You can modify the file name in "Save" dialog box, as shown in Figure 4-3.

Figure 4-3 Save Output File



4.2 Edit Constraints File

FloorPlanner supports constraints creation of I/O, primitive, group, resource reservation, global clock assignment, and reference voltage, etc. Constraints can be generated via Constraints menu, see [3.3.1 Menu Bar](#) for details.

Note!

Constraints can also be created by other ways; the following section mainly introduces how to generate constraints by dragging.

4.2.1 Constraints Examples

Take the user design counter.v for an instance to introduce how to create various constraints.

```
module counter1(out, cout, data, load, cin, clk, clko);
output [7:0] out;
output cout;
output clko;
input [7:0] data;
```

```
input load, cin, clk;
reg [7:0] out;
always @(posedge clk)
begin
    if (load)
        out = data;
    else
        out = out + cin;
end
assign cout = &out & cin;
wire clkout;
CLKDIV clkdiv_inst (
    .CLKOUT(clkout),
    .HCLKIN(clk),
    .RESETN(1'b1),
    .CALIB(1'b0)
);
defparam clkdiv_inst.DIV_MODE = "2";

DCS dcs_inst (
    .CLKOUT(clko),
    .CLKSEL(4'b0000),
    .CLKIN0(clkout),
    .CLKIN1(clkout),
    .CLKIN2(clkout),
    .CLKIN3(clkout),
    .SELFORCE(1'b0)
);
```



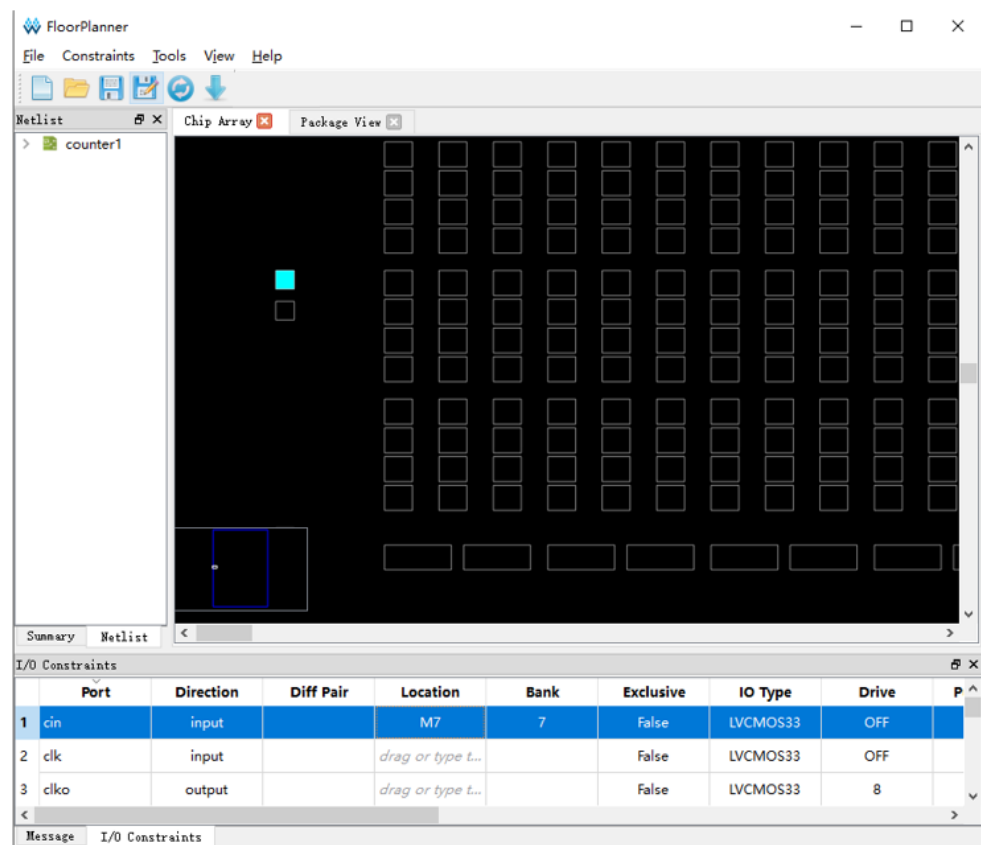
```
defparam dcs_inst.DCS_MODE = "RISING";
endmodule
```

4.2.2 Edit I/O Constraints

Drag to Chip Array to create I/O constraints.

1. Click I/O Constraints to zoom in Chip Array to macrocell mode.
2. Select Port "cin" and drag it to "M7" in Chip Array, as shown in Figure 4-4.
3. Port "cin" location is displayed as M7.

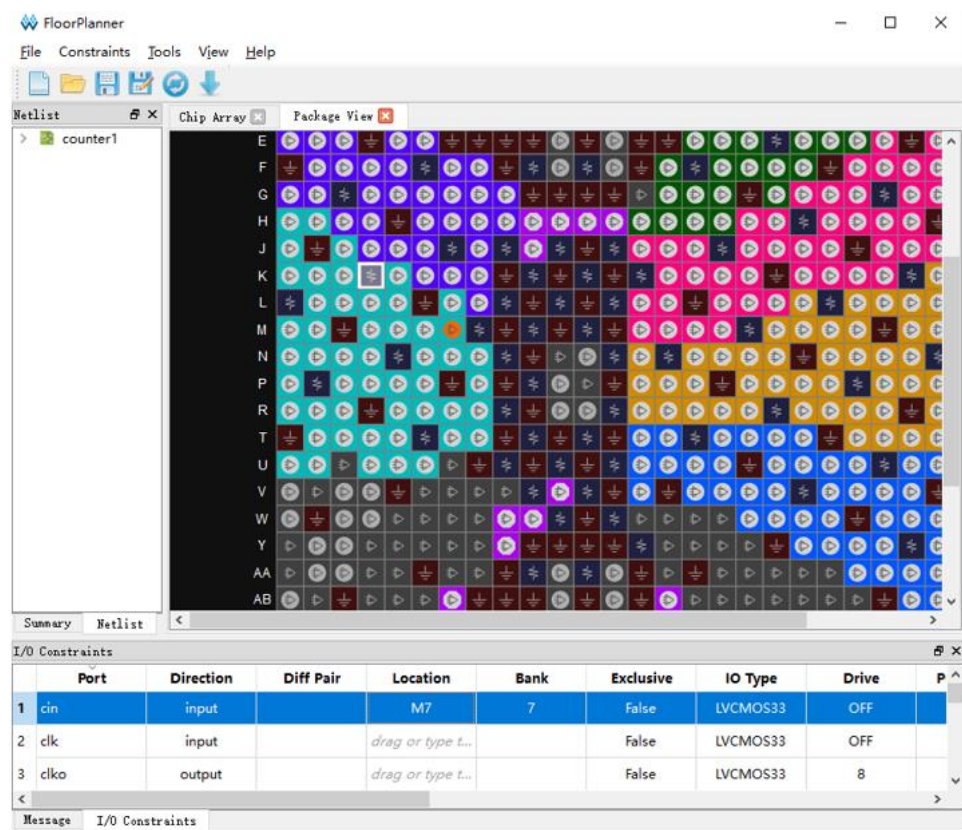
Figure 4-4 Drag to Chip Array to Create I/O Constraints



Drag to Package View to create I/O constraints.

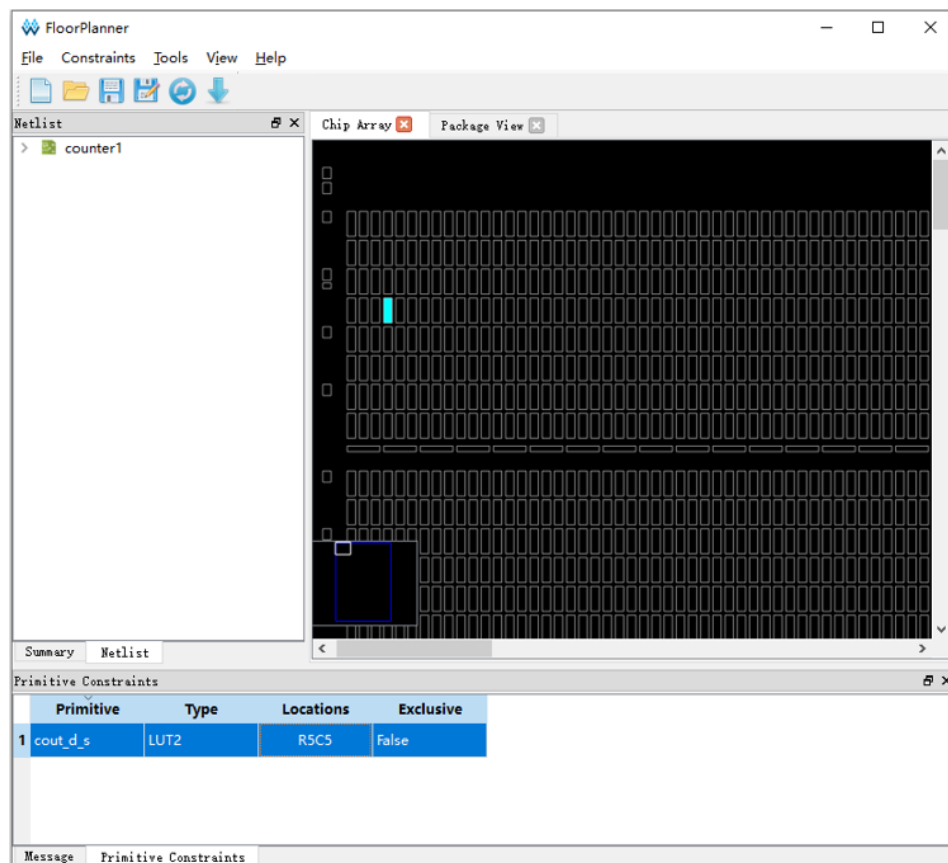
1. Click IO Constraints.
2. Select Port "cin" and drag it to "M7" in Package View, as shown in Figure 4-5.
3. Location for Port "cie" is displayed as M7.

Figure 4-5 Drag to Package View to Create I/O Constraints



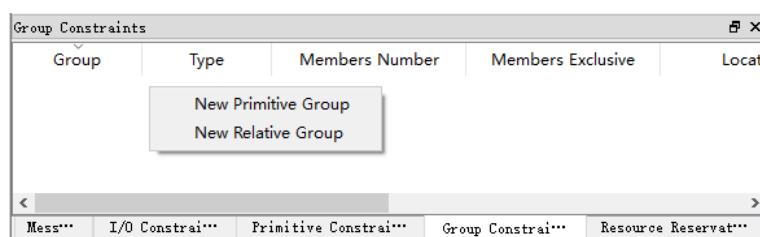
4.2.3 Edit Primitive Constraints

1. You can right-click the menu in "Primitive Constraints" and select "Select Primitives", then "Select Primitives" dialog box pops up. You can select Primitive "cout_d_s" and click "OK".
2. Select the created primitive constraints and drag it to "R5C5" in Chip Array, as shown in Figure 4-6.
3. Location for primitive "cout_d_s" is displayed as R5C5.

Figure 4-6 Drag to Chip Array to Create Primitive Constraints

4.2.4 Edit Group Constraints

As shown in Figure 4-7, you can create Primitive Group and Relative Group by right-clicking in Group Constraints.

Figure 4-7 Group Constraints Right-clicking

Create Primitive Group Constraints

1. Right-click "Group Constraints" and click "New Primitive Group", then "New Primitive Group" pops up.
2. Enter "grp1" and click "+", then "Select Primitives" dialog box pops up.
3. Select "n17_s0" and "cout_d_s"; click "OK", then add them to Members list.

4. Enter "R5C10" in "Locations", as shown in Figure 4-8.
5. Click "OK" in "New Primitive Group" dialog box to create primitive group constraints, as shown in Figure 4-9.

Figure 4-8 Create Primitive Group Constraints

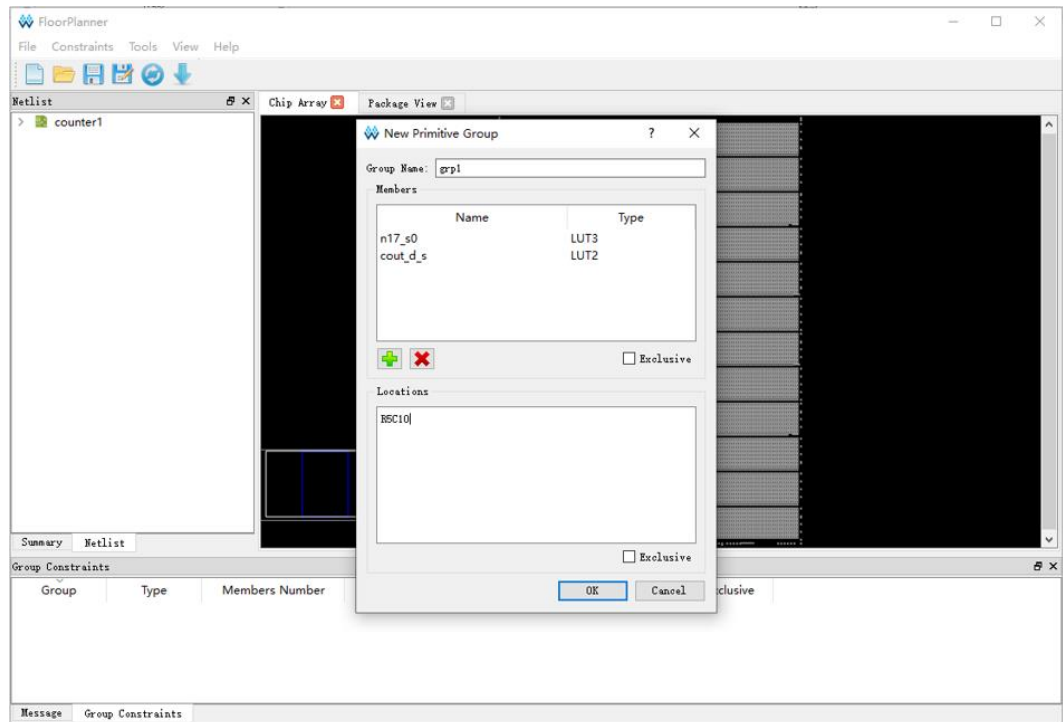
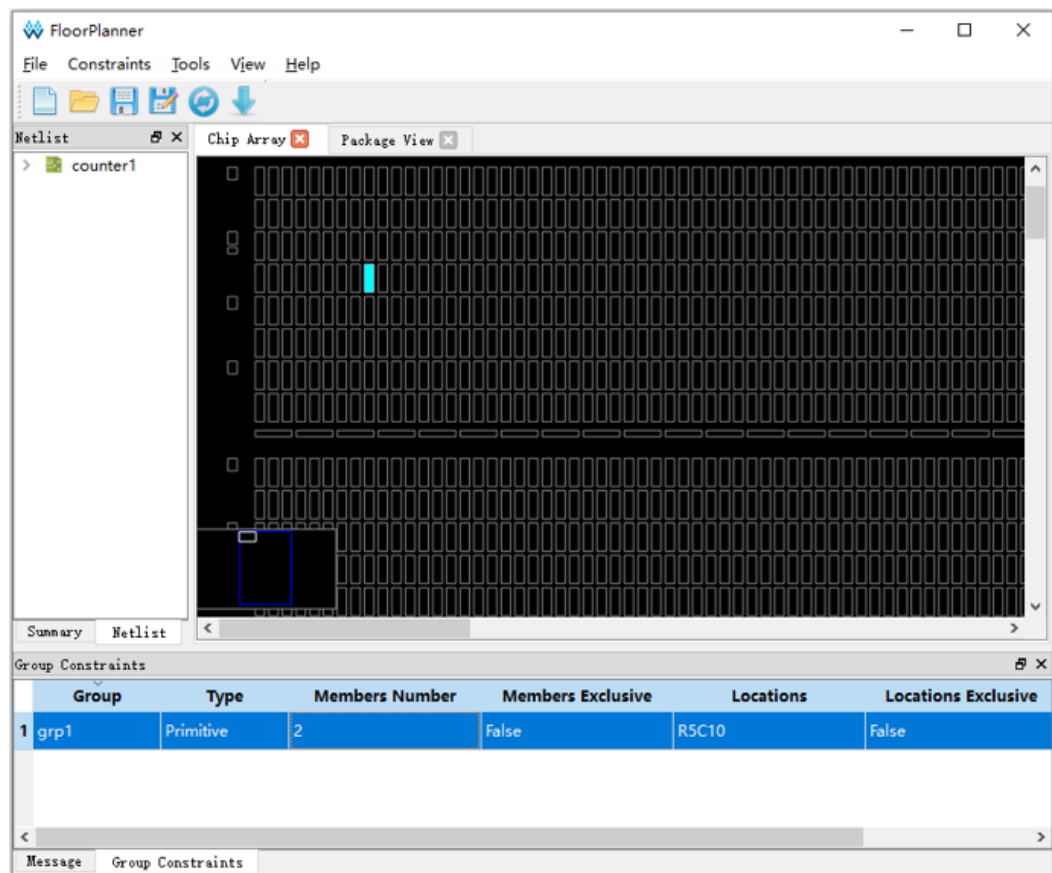


Figure 4-9 Primitive Group Constraints

**Note!**

The location in Primitive Group Constraints can only be entered manually or copied from Chip Array, and cannot be generated by dragging

Create Relative Group Constraints

1. Right-click "Group Constraints" and click "New Relative Group", and "New Relative Group" pops up.
2. Enter "rel_grp" and click "+", and "Select Primitives" dialog box pops up.
3. Select the primitives "cout_0_s1" and "cout_1_s1" in "Select Primitives" and click "OK", then add them to the Member list.
4. Add "R0C0" and "R4C5" to the Primitives, as shown in Figure 4-10.
5. Click "OK" in "New Relative Group" to create relative primitive group constraints, as shown in Figure 4-11.

Figure 4-10 Create Relative Group Constraints

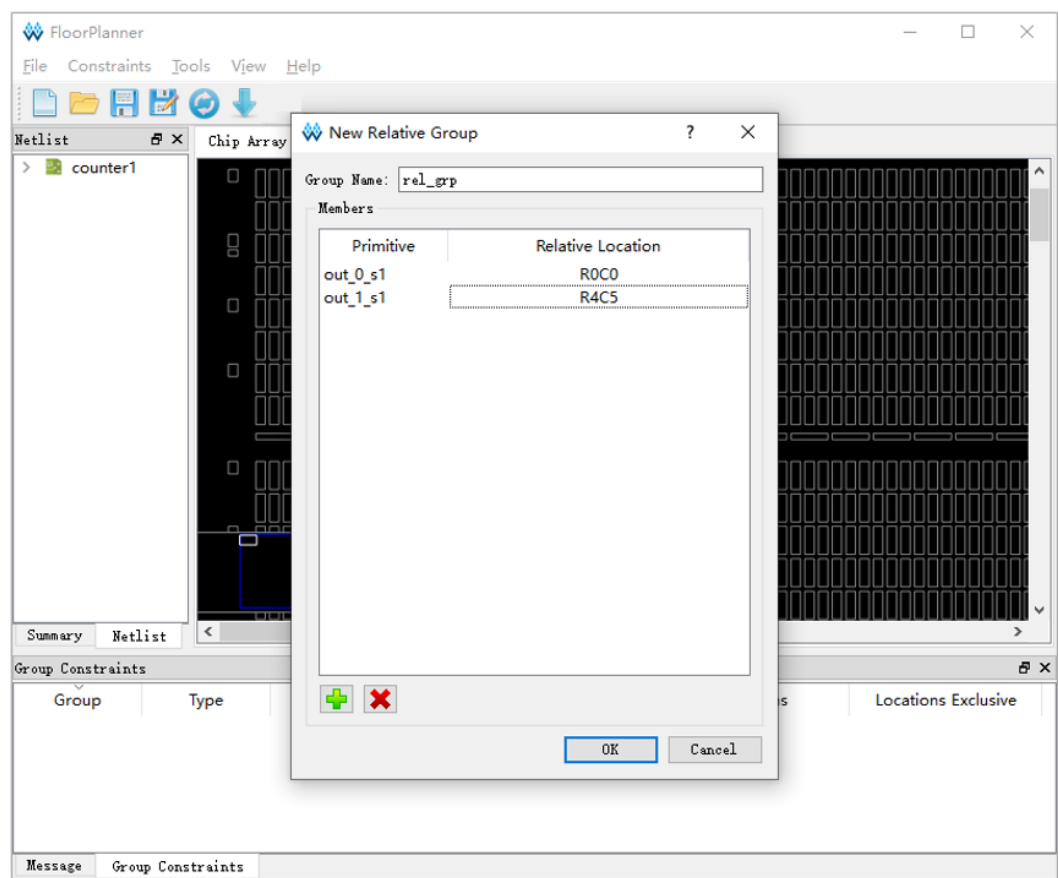
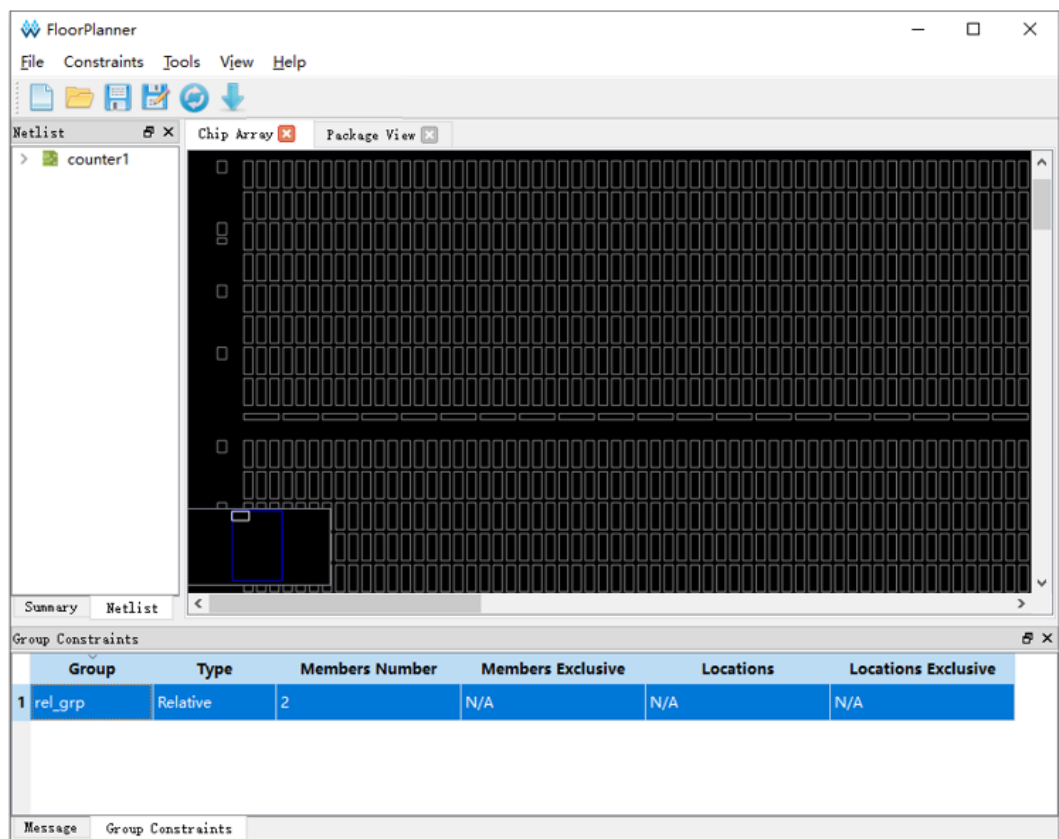


Figure 4-11 Relative Group Constraints



4.2.5 Edit Resource Reservation Constraints

1. Right-click "Resource Reservation" and click "Reserve Resources" to add resource reservation constraints, as shown in Figure 4-12.
2. Select the created resource reservation constraints and drag it to a location in Chip Array. As shown in Figure 4-13, drag to BSRAM_R10[1] to generate constraints.

Figure 4-12 Create Resource Reservation

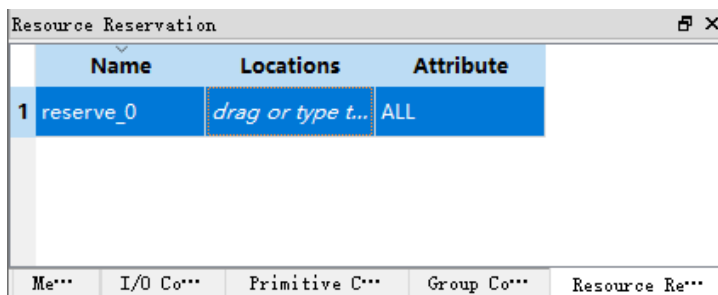


Figure 4-13 Resource Reservation

Resource Reservation		
	Name	Locations
1	reserve_0	BSRAM_R10[1]
		Attribute
		ALL
Me*** I/O Co*** Primitive C*** Group Co*** Resource Re***		

4.2.6 Edit Clock Net Constraints

1. Right-click "Clock Net Constraints" and select "Clock Net Constraints", then "Clock Net Constraints" dialog box pops up.
2. Click "+" and "Select Net" dialog box pops up. Select a Net and click "OK".
3. Select clock type and set signal, as shown in Figure 4-14.
4. Click "OK" to add constraints to Clock Net Constraints, as shown in Figure 4-15.

Figure 4-14 Create Clock Assignment Constraints

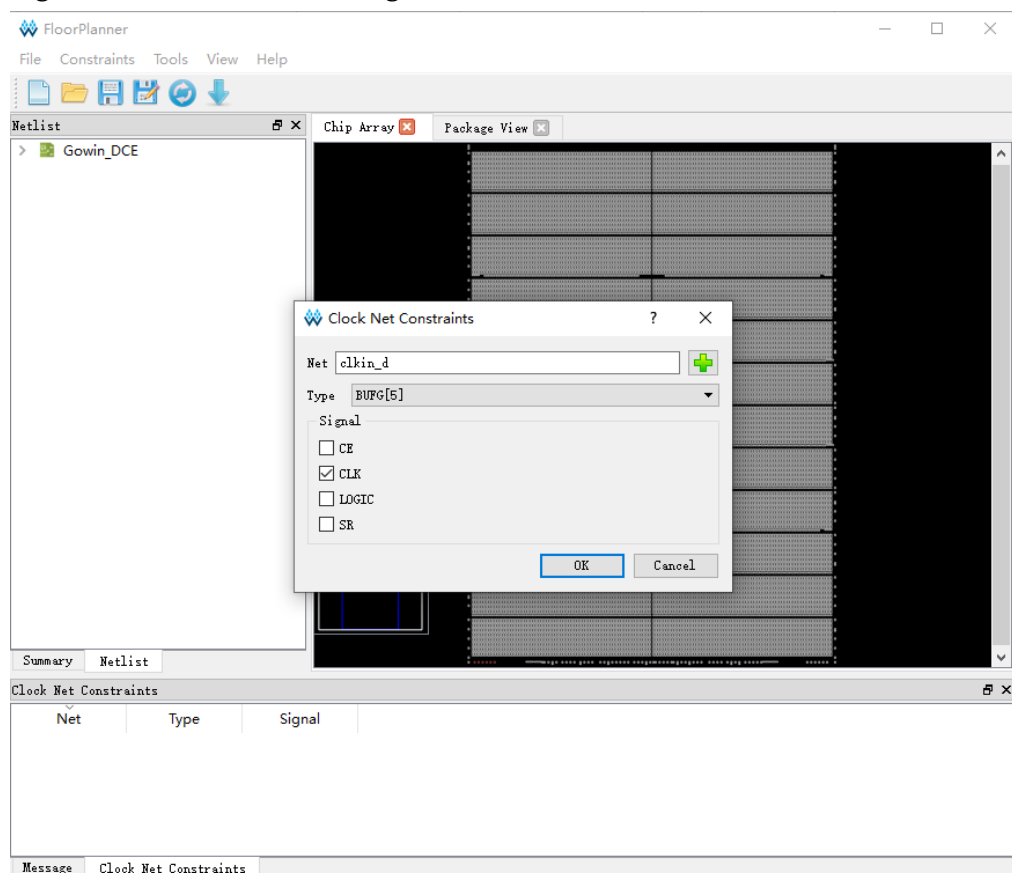
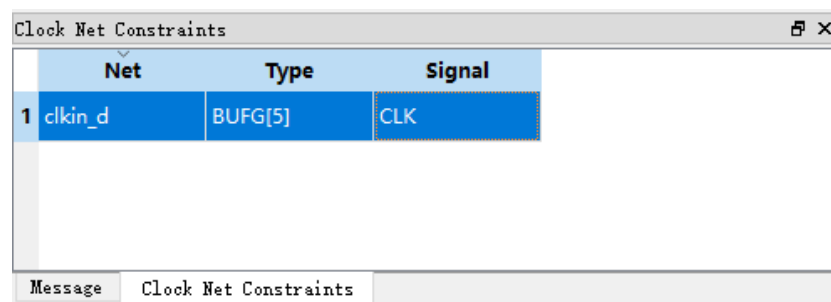


Figure 4-15 Clock Assignment Constraints



4.2.7 Edit GCLK Primitive Constraints

GCLK Primitive Constraints only supports constraints for DCS and DCE.

The steps to create GCLK Primitive Constraints are as follows:

1. In the GCLK Primitive Constraints editing window, right-click and select "Select GCLK Primitive", and the GCLK Primitive Constraints dialog box pops up.
2. Click "+" to open "GCLK Primitive" dialog box; select Instance, and click "OK".
3. Select the global clock constraint location using "Position", as shown in Figure 4-16.
4. Click "OK" in the GCLK Primitive Constraints dialog box, and the constraint will be added, as shown in Figure 4-17.

Figure 4-16 Create GCLK Primitive Constraints

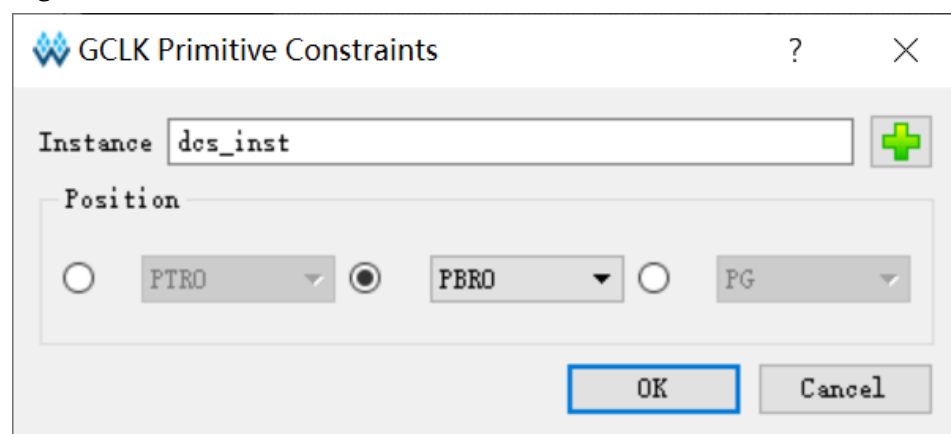


Figure 4-17 GCLK Primitive Constraints

GCLK Primitive Constraints		
	Instance	Type
1	dc5_inst	DCS
		PBR0
Message GCLK Primitive Constraints		

4.2.8 Edit HCLK Primitive Constraints

HCLK Primitive Constraints only supports Instance of CLKDIV and DLLDL.

The steps to create HCLK Primitive Constraints are as follows:

1. In the HCLK Primitive Constraints editing window, right-click and select "Select HCLK Primitive", and the HCLK Primitive Constraints dialog box pops up.
2. Click "+" to open "HCLK Primitive" dialog box; select Instance, and click "OK".
3. Select the high-speed clock constraint location using "Position", as shown in Figure 4-18.
4. Click "OK" in the HCLK Primitive Constraints dialog box, and the constraint will be added, as shown in Figure 4-19.

Figure 4-18 Create HCLK Primitive Constraints

HCLK Primitive Constraints

Instance:

Position:

☐ LEFTSIDE[0]
 ☐ RIGHTSIDE[0]
 ☒ BOTTOMSIDE[0]

OK Cancel

Figure 4-19 HCLK Primitive Constraints

Instance	Type	Positions
1 clkdiv_inst	CLKDIV	BOTTOMSIDE[0]

Message HCLK Primitive Constraints

4.2.9 Edit Vref Constraints

Drag to Chip Array to create Vref Constraints.

1. Right-click Vref Constraints and select "Define Vref Driver" to add the constraints to Vref Constraints, as shown in Figure 4-20.
2. Zoom in Chip Array to macrocell mode. Select the created Vref Constraints and drag it to B5 in Chip Array. The location of the Vref Constraints is displayed as "B5", as shown in Figure 4-21.

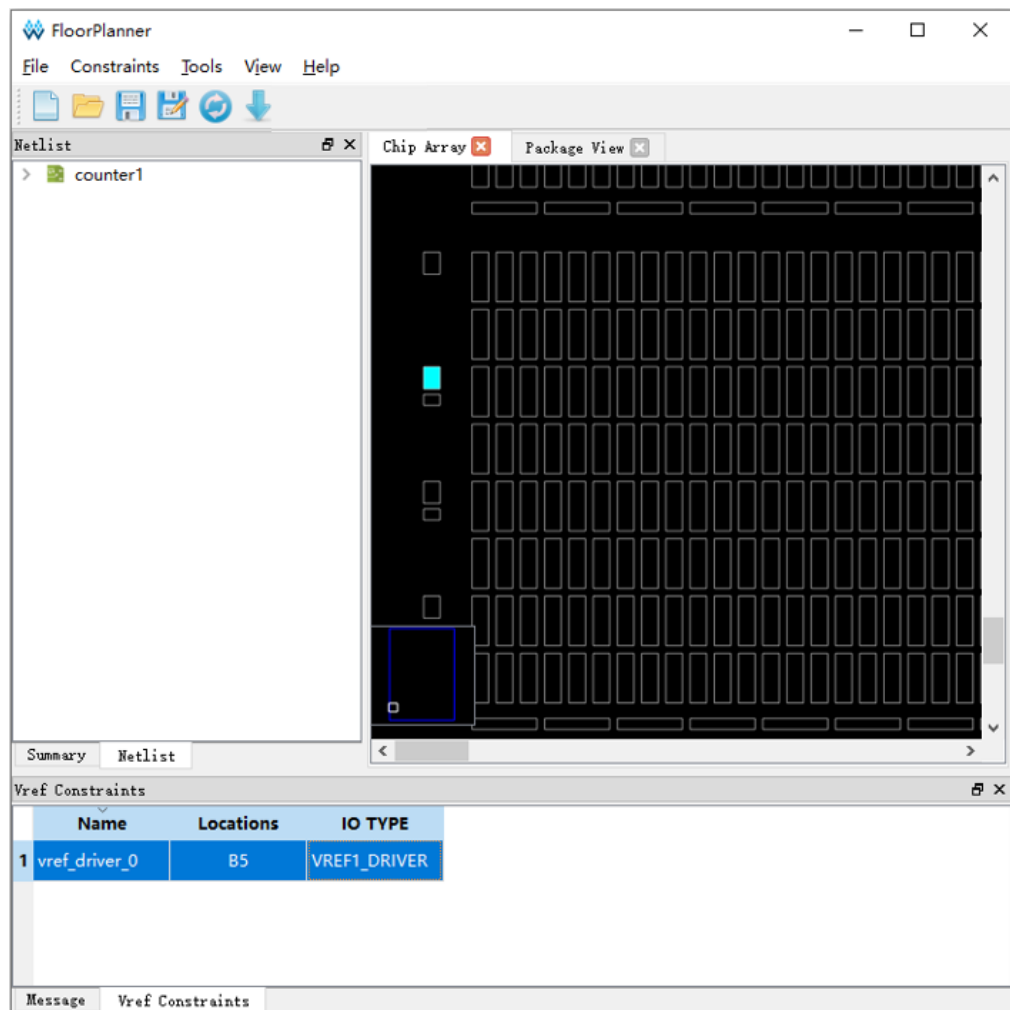
Figure 4-20 Create Vref Constraints

Name	Locations	IO TYPE
1 vref_driver_0	drag to set	VREF1_DRIVER

Message Vref Constraints

You can customize Vref constraint name, but duplicate names are not allowed; if there are duplicate names, you will be prompted, as shown in Figure 4-23.

Figure 4-21 Drag to Chip Array to Generate Vref Constraints Location



Drag to Package View to create Vref constraints.

1. Right-click Vref Constraints and select "Define Vref Driver" to add the constraints to Vref Constraints, as shown in Figure 4-20.
2. Select the newly created Vref Constraints and drag it to "B5" in Package View. The location of the Vref Constraints is displayed as "B5", as shown in Figure 4-22.

Figure 4-22 Drag to Package View to Generate Vref Constraints Location

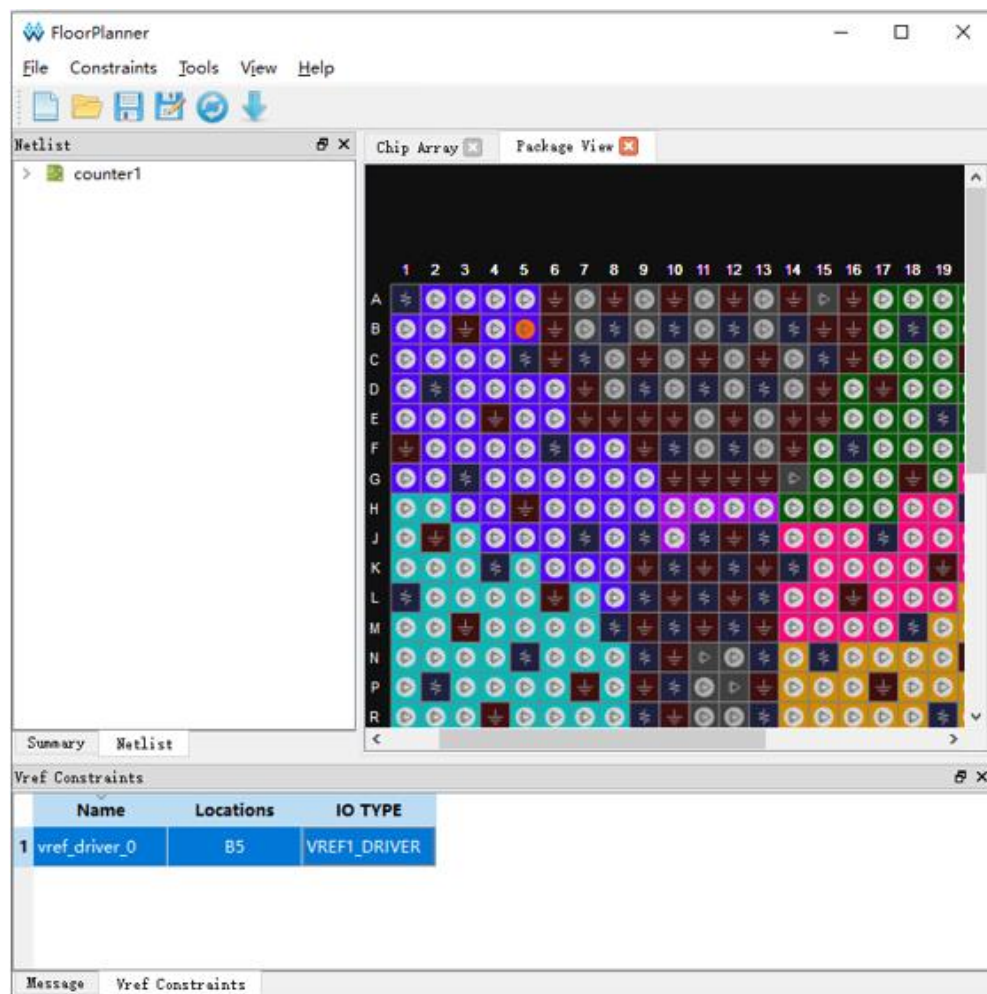


Figure 4-23 Duplicate Names Prompt

